Spiri

SpiriCamera

Release 0.1

Alex Davies

Nov 05, 2026

CONTENTS

1 Getting Started **3**
- 1.1 Installation . . . 3
- 1.2 Configuration . . . 3
- 1.3 Camera Usage . . . 3
- 1.4 Quick Start . . . 9
- 1.5 Next Steps . . . 9

2 Frame Metadata (EXIF) **11**
- 2.1 Reading Tags . . . 11
- 2.2 Multiple Tag Providers . . . 11
- 2.3 Example: Tagging Frames With GPS . . . 12
- 2.4 Reading Tags From Raw JPEG Bytes . . . 13

3 Drawing on things in the world **15**
- 3.1 The camera needs a place . . . 15
- 3.2 `view` . . . 16
- 3.3 What gets drawn . . . 16
- 3.4 Examples . . . 16
- 3.5 What it doesn't tell you . . . 17

4 Architecture **19**
- 4.1 Domain split: camera vs. sources . . . 19
- 4.2 Camera as a SpiriSynq object . . . 19
- 4.3 Adding a new source handler . . . 20
- 4.4 The overlay layer . . . 22
- 4.5 The debug UI's bandwidth and latency readout . . . 23
- 4.6 WebRTC egress, ingest, and WHEP . . . 24
- 4.7 Where things are tested . . . 25

5 Tutorial: YOLO detection boxes as a live overlay **27**
- 5.1 Running it against the NiceGUI debug UI . . . 27
- 5.2 How it fits together . . . 29
- 5.3 How many lines is this, really . . . 32
- 5.4 Cleaning up . . . 32

6 API Reference **33**

6.1 Classes 33
6.2 Modules 58

7 Revision history **121**
7.1 Issuing a revision 121

Python Module Index **123**

Index **125**

Document control

Document number	**SPIRI-SPIRICAMERA-001**
Revision	**0.1**
Status	DRAFT – not for operational use
Effective date	2026-01-01
Issued by	Alex Davies

Uncontrolled when printed.

Work with Cameras in SpiriSynq

CHAPTER

ONE

GETTING STARTED

This guide walks you through installing, configuring, and running SpiriCamera.

1.1 Installation

Add SpiriCamera to your project:

```
uv add SpiriCamera
```

Or install from source:

```
uv sync
uv lock
```

1.2 Configuration

SpiriCamera runs with sensible defaults. If you need to customize, set environment variables with the `SPIRI-CAMERA_` prefix:

```
export SPIRICAMERA_ENVIRONMENT=production
export SPIRICAMERA_LOG_LEVEL=DEBUG
```

Or create a `.env` file in your project root. See the *API Reference* for all available options.

1.3 Camera Usage

SpiriCamera provides a `Camera` class for live frame capture. A camera is a `SyncableObject`: it behaves like a small daemon that owns a device, captures in the background, and publishes each frame over SpiriSynq — not a handle you construct, read from synchronously, and throw away. The protocol detail of reaching a device lives in the source handlers under *`SpiriCamera.sources`*.

There are two roles, and most code only ever needs one of them:

- **Authoritative** — the process that actually owns the device (typically `SpiriCamera run`, or something like it). It constructs a `Camera` normally, `start()`s it, and lets the background thread do the work.

- **Mirror** — everyone else. Rather than constructing a second `Camera` pointed at the same device, get a handle onto the *existing* one over SpiriSynq and read its synced fields as they arrive.

1.3.1 Consuming an existing camera (the common case)

Attach to a camera another process is already running by its SpiriSynq topic:

```
from SpiriCamera import Camera

cam = Camera.from_topic("some-host/dev-video0")
```

This gives you a mirror: same synced fields (`image`, `status`, `running`, …), kept up to date by SpiriSynq, but it never opens a device locally — it exists to observe, not to capture. There is no `read()` to call. Get frames one of two ways:

Poll, when you're already on a timer or a render loop (this is what the debug UI does, ticking in step with `max_framerate`):

```
frame_jpeg = cam.image  # the most recent frame, updated for you in the␣
↪background
```

Or subscribe, when you want to react the moment a new frame (or any other field) lands, via the psygnal event every synced field gets:

```
def on_frame(frame_jpeg: bytes) -> None: ...


cam.events.image.connect(on_frame)
```

Both approaches work the same way whether `cam` is a mirror or the authoritative camera itself — `image` is just a field, and it changes the same way either way.

1.3.2 Building on an existing camera

A mirror is a view of someone else's camera: its settings and overlays are that camera's, and changing them changes it for everyone. To get a *different* stream out of it — smaller, with your own overlays, at your own quality or framerate — use it as the source of a camera of your own instead:

```
from SpiriCamera import Camera

small = Camera(
    "spirisynq://some-host/dev-video0",
    max_width=640,
    max_height=360,
    quality=60,
)
small.start()
```

`spirisynq://` takes the same topic `Camera.from_topic` does, absolute or relative to this host's base topic: `spirisynq://./camera1` is this host's `camera1`. Each upstream frame is decoded and scaled down to fit

inside `max_width` x `max_height` (keeping its aspect ratio; it never scales up), then goes through this camera like any source's frame: its own `overlay_widgets` are baked in, it is re-encoded at its own `quality`, and it is published on its own topic. The upstream is untouched, so any number of these can hang off one device. Frame tags carry over too — see *Frame Metadata (EXIF)*.

The upstream does not have to exist yet, and may come and go: the camera waits for the topic, and reconnects when an upstream that shut down, crashed, or restarted is back — see "Waiting for a Source" below. An upstream that is merely stopped is kept; its last frame stays up until it starts again.

1.3.3 Running a device yourself

If your process *is* the one meant to own the device — a standalone script, or something like the `SpiriCamera run` CLI command — construct a `Camera` normally and start it. Leave `start()` on its default `background=True`: the capture thread runs its own loop, paced to `max_framerate`, for as long as the camera is running. Don't drive it frame-at-a-time yourself — a `Camera` is meant to be treated like a small daemon, not a handle you poll by calling `read()` in your own loop.

```
from SpiriCamera import Camera

cam = Camera("/dev/video0", quality=85)
cam.start()  # opens the device, begins the background capture thread

frame_jpeg = cam.image  # whatever the background thread has captured so far

cam.stop()
```

or as a context manager, which starts on entry and stops on exit:

```
with Camera("/dev/video0", quality=85) as cam:
    frame_jpeg = cam.image
```

To change the rate frames are captured at, set `cam.max_framerate` (live — takes effect on the next frame, no restart needed) rather than calling `read()` less often. `start(background=False)` exists — it opens the device without the capture thread, leaving you to call `read()` yourself — but that's an edge case for tests and one-off scripts, not the pattern to reach for; ordinary code, including anything long-running, should stay on the default background thread and read `cam.image` or connect to `cam.events.image`.

1.3.4 Inspecting a Source

Source strings are resolved by handlers that register themselves. To inspect one without opening any hardware:

```
from SpiriCamera import describe_source, known_schemes

info = describe_source("rtsp://host/stream")
print(info.scheme, info.target, info.handler)

print(known_schemes())
```

A camera whose source string cannot be resolved does not raise on construction; the reason lands in `cam.source.error` and `start()` is what refuses. This keeps a camera safe to bind to a live-edited input field.

1.3.5 Network Streams

```
# RTSP stream
cam = Camera("rtsp://admin:pass@192.168.1.100:554/stream")
cam.start()

# HTTP image stream (MJPEG)
cam = Camera("http://10.0.0.5/image.jpg")
cam.start()
```

1.3.6 Files

A video file or a still image can stand in for a camera. Files are just paths — absolute or relative, no scheme needed:

```
cam = Camera("/srv/clips/approach.mp4")  # loops at the end
cam = Camera("./pattern.png")  # a steady picture
```

Use `file://` when a path would otherwise read as a URL. POSIX collapses repeated slashes, so `rtsp://clip.mp4` is a real path to `clip.mp4` inside a directory named `rtsp:` — and a stream URL. The stream wins; `file://rtsp://clip.mp4` says you meant the file.

1.3.7 Local Devices

A camera index, a device node, or an explicit scheme:

```
cam = Camera("0")  # camera index, the OpenCV convention
cam = Camera("/dev/video0")  # device node
cam = Camera("v4l:///dev/video5")  # not plugged in yet
```

A schemeless path is recognised as a camera if the filesystem says it is one, or if it is named like one — `/dev/video<N>`, or anything under `/dev/v4l/` (the `by-id` links are the robust way to name a USB camera). So `/dev/video0` works even before the device is plugged in, while `/dev/null` and a half-typed `/dev/vi` do not. `v4l://` accepts any path.

1.3.8 Waiting for a Source

A started camera does not give up on a source that is not there. A device that is unplugged, or a `spirisynq://` topic nobody publishes yet, leaves the camera `running` with a status of `waiting for source: <reason>`, retrying about once a second until it works. The same happens if the device is unplugged or the upstream camera goes away mid-stream: it waits, and picks up again when it is back. `SpiriCamera run` therefore keeps running through a camera being replugged or a host rebooting. Only `start(background=False)` — scripted use with no capture thread to wait on — still raises instead.

1.3.9 Knowing What It Is Doing

`camera.status` says what the camera is doing in words — `running`, `stopped`, waiting, or the reason it is none of those:

```
cam = Camera("/dev/vi")  # half-typed
cam.status  # "Unrecognised source: '/dev/vi'. Supported: ..."

cam = Camera("v4l:///dev/video99")
cam.start()  # does not raise
cam.status  # "waiting for source: Failed to open capture source: '/dev/video99
↪'"
```

`running` says whether the camera is trying; `status` says whether frames are actually flowing, and why not when they are not.

1.3.10 Frame Tags

Each captured frame is a JPEG with its own metadata baked into its EXIF: when it was taken, what took it, and anything you care to add — see the *Frame Metadata (EXIF)* guide for how to read tags back, add your own (GPS included), and share the tagging job between more than one piece of software.

1.3.11 Overlays

A camera can bake a small SVG "HUD" onto every frame before it is encoded — a resolution/framerate readout, a battery level, anything templated from live data. The built-in metrics overlay is the quickest way to see this working:

```
from SpiriCamera import Camera
from SpiriCamera.overlay import camera_metrics_widget

cam = Camera("/dev/video0")
widget = camera_metrics_widget(cam)  # publishes its own topic
cam.overlay_widgets[widget.synq_topic] = {}  # opt this camera into it
cam.start()
```

`cam.image` now includes the overlay, composited fresh on every frame. Removing the entry (`del cam.overlay_widgets[widget.synq_topic]`) drops it again immediately — no restart needed.

A widget you author yourself is a *`HudWidget`*: an SVG template, MiniJinja-rendered against the current frame's `exif` tags, its actual `frame` (as-received width/height/framerate), and any other live SpiriSynq object it declares under `objects`:

```
from SpiriCamera.overlay import HudWidget

widget = HudWidget(
    synq_topic="my-hud",
    synq_authoritive=True,
    anchor="top_right",
```

(continues on next page)

(continued from previous page)

```
    svg_template="""
        {# object: battery = azrael/battery_monitor #}
        <svg xmlns="http://www.w3.org/2000/svg" width="160" height="30">
          <text x="4" y="20" font-size="16" fill="white">
            {{ objects.battery.voltage }}V @ {{ frame.framerate }}fps
          </text>
        </svg>
    """,
)
cam.overlay_widgets[widget.synq_topic] = {}
```

The `{# object: battery = ... #}` comment declares both the alias used in the template and the default topic it mirrors — every camera that opts into this widget gets that binding unless it overrides it for itself:

```
cam.overlay_widgets[widget.synq_topic] = {
    "bindings": {"battery": "some-other-host/battery_monitor"},
    "x": 2,
    "y": 2,  # percent-of-frame position override, this camera only
}
```

A widget you author is not owned by any one camera — publish it once and any number of cameras can mirror it, each with its own binding and placement overrides. See *`SpiriCamera.overlay`* (or the *overlay architecture notes*) for the full templating rules, including why `frame` is a separate namespace from `objects`.

1.3.12 Configuration

Camera parameters are configured via environment variables:

```
export SPIRICAMERA_SOURCE="rtsp://host/stream"
export SPIRICAMERA_QUALITY=90      # JPEG quality [1-100]
export SPIRICAMERA_FRAME_WIDTH=1280 # Capture width
export SPIRICAMERA_FRAME_HEIGHT=720 # Capture height
export SPIRICAMERA_FRAMERATE=30     # Capture framerate
```

Or passed directly to the `Camera` constructor:

```
cam = Camera(
    source="/dev/video0",
    quality=85,
    max_width=1280,
    max_height=720,
    max_framerate=30,
)
```

1.3.13 CLI Usage

List the registered source handlers and the schemes they claim:

```
SpiriCamera sources
```

Validate a source and report what the device can do:

```
SpiriCamera validate /dev/video0
```

Capture frames to a file:

```
SpiriCamera capture "rtsp://host/stream" --frames=10 -o frames.jpg
```

Run live capture, publishing to SpiriSynq until interrupted:

```
SpiriCamera run /dev/video0
```

The source argument is optional everywhere; without it, `SPIRICAMERA_SOURCE` is used.

1.4 Quick Start

Run the CLI:

```
SpiriCamera --help
```

Or run as a module:

```
python -m SpiriCamera
```

For verbose output:

```
SpiriCamera run --verbose
```

1.5 Next Steps

- Read the *API Reference* for programmatic details
- Check the pyproject.toml for build and dependency configuration
- See the Docker compose files for deployment options

CHAPTER

TWO

FRAME METADATA (EXIF)

Each captured frame is a JPEG with its own metadata baked into its EXIF: when it was taken, what took it, and anything you care to add — a GPS fix, a mission name, a battery level. This page covers reading that metadata back, adding your own, and sharing the job between more than one piece of software. For the underlying module (`embed`, `extract`, `is_jpeg`, `image_size`) used directly on raw JPEG bytes, see *SpiriCamera.exif*.

2.1 Reading Tags

```
cam = Camera("/dev/video0")
cam.exif_update(mission="probe-1")
cam.start()

cam.exif_tags  # {'timestamp': '1789...', 'make': ..., 'mission': 'probe-1'}
cam.exif_timestamp  # Unix timestamp of the frame currently in cam.image
```

`exif_tags` and `exif_timestamp` are read back *out of* `cam.image`, and so are `received_width`, `received_height` and `received_ratio`. None of them is sent over SpiriSynq — they are properties, not synced fields, so there is nothing there *to* send. That is on purpose: SpiriSynq publishes each field independently and promises nothing about two of them arriving together, so a timestamp sent beside an image is a timestamp a peer can read against the wrong image. Travelling inside the JPEG, they cannot come apart from it — a peer that receives a frame fills them in for itself, exactly:

```
mirror.image = frame  # the only thing that crossed the network
mirror.exif_timestamp  # when the far end took it
mirror.received_width  # what it actually sent
```

2.2 Multiple Tag Providers

More than one piece of software may want to tag the same camera's frames — a logger, a telemetry service, a human poking at the debug UI. `exif_set_tags` gives each one its own namespace, so setting one never touches another's:

```
cam.exif_set_tags("telemetry", {"battery": "88"})
cam.exif_set_tags("logger", {"build": "42"})
```

(continues on next page)

(continued from previous page)

```
cam.exif_clear_tags("telemetry")  # only "battery" goes away
```

Calling `exif_set_tags` again for the same provider *replaces* that provider's tags rather than merging into them — drop a key by leaving it out next time. A value of `""` is kept rather than dropped, which is how a provider suppresses one of the built-in tags (say, omitting `source` from a frame headed somewhere public). Providers merge over the built-ins in sorted-name order, so the result does not depend on which one registered first; two providers naming the *same* tag still resolve in that order, which is worth avoiding with a distinguishing prefix rather than relying on.

`exif_update(**tags)` is a shorthand for the common case of one caller adding a tag or two with no provider name of its own — it merges into a shared default bucket, and a value of `""` there removes just that tag:

```
cam.exif_update(mission="probe-1")
cam.exif_update(mission="")  # removes it; other tags untouched
```

Two *different* callers both using `exif_update` still share that one bucket and can still overwrite each other; once that matters, give each one a provider name and use `exif_set_tags` instead.

To attach something the camera itself can compute, override the method that builds the built-in tags — this still gets whatever providers have set merged on top:

```
class SurveyCamera(Camera):
    def exif_tags_for_frame(self, frame):
        tags = super().exif_tags_for_frame(frame)
        tags["gps"] = f"{self.fix.latitude},{self.fix.longitude}"
        return tags
```

Tag names are free-form; the whole set round-trips through the EXIF `UserComment` field. Names that have a real EXIF equivalent — `make`, `model`, `serial_number`, `software`, `description`, `datetime` — are also written to that tag, so ordinary image tools show something sensible.

A camera fed by another camera (a `spirisynq://` source, see *Getting Started*) starts from the tags the upstream frame already carried, so a GPS fix added upstream is still there after the re-encode. Its own built-ins then replace the upstream's `source`, `topic` and `software` — those describe whoever produced *these* bytes — but the upstream's `timestamp` is kept, since that is when the picture was actually taken. Providers merge on top of both, as usual.

Because a tag records a capture time, two captures of an unchanging scene produce different bytes. Set `exif_enabled = False` for byte-identical frames, which psygnal then suppresses rather than republishing.

2.3 Example: Tagging Frames With GPS

A GPS fix is exactly the kind of thing that changes on its own schedule, independent of the frame rate, so the natural way to attach it is `exif_update` from whatever callback your GPS source already gives you — not something computed inside `Camera` itself, and not something worth a subclass for:

```
cam = Camera("/dev/video0")
cam.start()


def on_gps_fix(lat: float, lon: float) -> None:
    cam.exif_update(gps_lat=f"{lat:.6f}", gps_lon=f"{lon:.6f}")


gps.events.fix.connect(on_gps_fix)  # or however your GPS source notifies you
```

Every frame captured after that carries the most recent fix — there is no need to re-call `exif_update` per frame, since `exif_tags_for_frame` re-merges whatever the default bucket currently holds into *every* frame it builds. A fix that goes stale (GPS lost) should be cleared the same way, with an empty string, rather than left showing a last-known position as if it were current:

```
cam.exif_update(gps_lat="", gps_lon="")  # drop both; other tags untouched
```

If your GPS source lives inside the `Camera` subclass itself — an onboard receiver polled on the same cadence as capture, say — override `exif_tags_for_frame` instead, so the fix is read fresh for the frame actually being encoded rather than from whatever `exif_update` last pushed:

```
class SurveyCamera(Camera):
    def exif_tags_for_frame(self, frame):
        tags = super().exif_tags_for_frame(frame)
        fix = self.gps.read()
        if fix is not None:
            tags["gps_lat"] = f"{fix.latitude:.6f}"
            tags["gps_lon"] = f"{fix.longitude:.6f}"
        return tags
```

Either way, `gps_lat`/`gps_lon` are free-form tag names, not a real EXIF GPS IFD entry — they round-trip through the `UserComment` JSON exactly like any other tag, but a tool that reads a JPEG's *native* GPS fields (only `make`, `model`, `serial_number`, `software`, `description` and `datetime` get written there; see `STANDARD_TAGS` in *SpiriCamera.exif*) will not see them. Read them back with `cam.exif_tags["gps_lat"]`, or `exif.extract(data)["gps_lat"]` on raw bytes via *SpiriCamera.exif*.

2.4 Reading Tags From Raw JPEG Bytes

`cam.exif_tags` is a convenience for when you already have a `Camera`. If all you have is JPEG bytes — a frame saved to disk, one pulled off an HTTP response, `mirror.image` read before you bothered to build a mirror at all — *SpiriCamera.exif* reads and writes the same tags directly, with no `Camera` involved:

```
from SpiriCamera import exif

data = open("frame.jpg", "rb").read()
```

(continues on next page)

(continued from previous page)

```
exif.is_jpeg(data)  # True — cheap check, just the magic bytes
exif.extract(data)  # {'timestamp': '1789...', 'make': ..., 'mission': 'probe-1
↪'}
exif.image_size(data)  # (1920, 1080) — read from the JPEG header, not decoded
```

`extract` never raises: bytes with no tags, damaged EXIF, or a format that carries none all read back as `{}`, so it is safe to call on whatever you happen to have. It reads the same `UserComment` JSON blob `Camera` writes, so an arbitrary tag name round-trips exactly; given a JPEG this package did not tag at all, it falls back to whatever standard EXIF fields (`Make`, `Model`, `DateTime`, …) the file happens to carry.

`embed` is the write side, and is what `Camera` calls internally on every captured frame:

```
tagged = exif.embed(data, {"mission": "probe-1", "operator": "alex"})
```

Tagging is a best effort layered on top of whatever encoding already happened: `embed` returns non-JPEG data unchanged rather than raising, and only raises `exif.ExifError` for a JPEG it could not tag. Calling it again replaces the EXIF wholesale rather than accumulating segments, so re-tagging a frame — or a mirror re-deriving its own tags from `image` — never leaves stale data behind.

CHAPTER

THREE

DRAWING ON THINGS IN THE WORLD

Most HUD widgets sit at a fixed place on the frame: a battery readout in a corner, a heading tape across the top. Some belong on *something in the picture*: a marker on home, on the region of interest, on where the rangefinder hit, on each detection. Those things are already in SpiriSynq's frame tree as `Position`s with poses in ECEF, so a widget can say which object it belongs on, and the camera works out where that is in the picture.

The widget's SVG never does the geometry. It draws itself around its own centre, and the camera places it. A widget opts in with one declaration in its template, next to its `{# object: #}` declarations. A widget that doesn't declare one works as it always has, and costs nothing extra.

Declaration	Draws	Template gets
`{# anchor: <alias> #}`	one copy, centred on the object	`view`: the object's place
`{# anchor: each <alias>.<field> #}`	one copy per object linked from `<field>`	`view` and `item`: each linked object's place and fields
`{# project: <alias>, ... #}`	one copy, on the frame, as usual	`view.<alias>`: each object's place, or `none`

Every alias still needs its `{# object: <alias> = <topic> #}` declaration, and a camera can rebind it with `overlay_widgets`' `"bindings"`, as for any other widget. A widget has at most one `anchor`, and either anchors or projects, not both.

3.1 The camera needs a place

To know where an object is in its picture, the camera needs its own pose and lens:

- **Its lens** is *`intrinsics`*, from settings, its source, or SpiriMav (see `set_intrinsics`).
- **Its pose:** a *`CameraBase`* is a `Position`. Mount it on whatever carries it, usually a gimbal, by setting `relative_to`. `SPIRICAMERA_RELATIVE_TO=mu-1/robot_gimbal1` does that for `SpiriCamera run`, with an identity mount: no offset, the gimbal's axes. For anything else, set `mount` over SpiriSynq.
- **A relay** (a `spirisynq://` source, such as a HUD variant of another camera) shows another camera's view. It follows that camera's pose and its lens, live, unless it has been given a place of its own.

Until a camera has both a pose and a lens, it draws no anchored widgets, and `view.<alias>` in a projecting widget is `none`.

A camera's own frame is FRD, like the rest of the tree: `x` out of the lens, `y` to the right of the picture, `z` down it. A camera on a gimbal with an identity mount looks along the gimbal's `x` axis, the way a rangefinder on that gimbal points.

3.2 `view`

`View`'s fields:

Field	
`x`, `y`	The object's pixel.
`r`	The radius, in pixels, of the circle holding the object's 90% accuracy cylinder (`Accuracy`), seen from here. `none` when the object's accuracy is unknown, or the cylinder reaches behind the camera.
`rang`	Metres from the camera to the object.
`edge`	Whether the object, or its circle, crosses the frame's edge.

The units are the ones the widget draws in:

- **For an anchored widget**, that means the widget's own pixels. These are the frame's pixels when overlays are burned in, and the canvas's when a browser renders them (`render_overlays_for_client`). So `<circle r="{{ view.r }}">` is the right size either way.
- **For a projecting widget**, they are the frame's pixels, the same as `frame.width` and `frame.height`. So the widget covers the frame with `width="100%" height="100%" viewBox="0 0 {{ frame.width }} {{ frame.height }}"` and draws at the pixels it's given.

3.3 What gets drawn

- **An anchored copy is drawn** while any of its circle is on the frame. Off the frame, behind the camera, or with no pose (a rangefinder with no return), it isn't drawn. A copy partly off the frame is clipped.
- **A projecting widget is always drawn.** An object in front of the camera but out of shot still has its pixel, off the frame, and the SVG's own clipping handles anything drawn there. An object behind the camera, or with no pose, is `none`. A line to it has no honest 2D end, so skip it:

```
{% if view.a and view.b %}<line x1="{{ view.a.x }}" ... />{% endif %}
```

- **For `anchor:` `each`**, the field is a list of `Link`s, such as a robot's `gimbals`. Each linked object is mirrored as the list changes. A new link shows a frame after the list does, once its object has been mirrored.

3.4 Examples

A marker on home, with its distance:

```
{# object: home = mu-1/robot_home #}
{# anchor: home #}
<svg xmlns="http://www.w3.org/2000/svg" width="80" height="44">
  <path d="M40 10 L48 22 L40 34 L32 22 Z" fill="none" stroke="white"␣
↪stroke-width="2"/>
  <text x="26" y="44" font-family="Miracode" font-size="13" fill="white">
    {{ view.range | round | int }} m
  </text>
</svg>
```

The widget is 80 px wide and 44 px tall, so its centre (40, 22) is the diamond's centre, and that is what lands on home.

A ring on each detection, sized to how sure it is of where it is:

```
{# object: detector = mu-1/detector #}
{# anchor: each detector.detections #}
{% set r = [view.r or 0, 8] | max %}
<svg xmlns="http://www.w3.org/2000/svg" width="{{ 2 * r + 4 }}" height="{{ 2 *␣
↪r + 4 }}">
  <circle cx="{{ r + 2 }}" cy="{{ r + 2 }}" r="{{ r }}" fill="none"␣
↪stroke="yellow" stroke-width="2"/>
</svg>
```

A line from where the rangefinder hit to the region of interest:

```
{# object: spot = mu-1/robot_gimbal1_range #}
{# object: roi = mu-1/robot_roi #}
{# project: spot, roi #}
<svg xmlns="http://www.w3.org/2000/svg" width="100%" height="100%"
     viewBox="0 0 {{ frame.width }} {{ frame.height }}">
  {% if view.spot and view.roi %}
  <line x1="{{ view.spot.x }}" y1="{{ view.spot.y }}" x2="{{ view.roi.x }}"␣
↪y2="{{ view.roi.y }}"
        stroke="white" stroke-dasharray="6 4"/>
  {% endif %}
</svg>
```

3.5 What it doesn't tell you

- **`r` is the object's own uncertainty, not the camera's.** It answers "where in the world could this be, as seen from here", not "where in this picture will it appear". These two differ when the object and the camera share an error. A rangefinder spot on the camera's own gimbal carries the robot's GPS error in its accuracy, but that error moves the camera too, so in the camera's own picture the spot sits on the axis however large `r` is. Draw `r` when the "where in the world" question is the one the widget answers.
- **The pose is the latest one,** not the pose at the instant the frame was taken: SpiriSynq poses carry no time yet. While a gimbal slews, an anchored widget trails the picture by however stale the pose is.

- **The lens is a pinhole,** with no distortion. Near the edge of a wide lens, a marker is some pixels off.
- **The camera's own mount is taken as exact:** a camera that is a few centimetres off its gimbal's axis, or not quite aligned with it, puts everything that far off.

CHAPTER

FOUR

ARCHITECTURE

This page is for anyone changing SpiriCamera itself, rather than just using a `Camera`. It covers the things that only live in scattered docstrings today: how the package is split into domains, how a `Camera` behaves as a SpiriSynq object, how to add a new source handler, and what the debug UI's bandwidth/latency readout actually measures.

4.1 Domain split: camera vs. sources

The package has one dependency direction, enforced by convention, not by tooling:

```
SpiriCamera.camera  ->  SpiriCamera.sources  ->  SpiriCamera.sources.base
```

- **`SpiriCamera.sources`** (and its handler modules — `v4l`, `network`, `file`, `testimage`) owns *how to reach a device*: parsing a source string, opening it, and handing back raw BGR `numpy` arrays. A handler never imports *`Camera`* and knows nothing about JPEG encoding, SpiriSynq, or EXIF.
- **`SpiriCamera.camera`** owns *what happens to a frame once you have one*: the capture thread, encoding to JPEG, tagging, and publishing via `SyncableObject`. It talks to a source only through the *`SourceBase`* contract.
- **`SpiriCamera.exif`** is used by `camera.py` to embed/extract tags, and by `ui.py` to read a frame's capture timestamp back out. It does not know about `Camera` or sources either.

If you find yourself importing `SpiriCamera.camera` from inside `SpiriCamera/sources/`, that's the layering breaking — a source should never need to know it's being driven by a `Camera`.

4.2 Camera as a SpiriSynq object

`Camera` is a `SyncableObject` (*`CameraBase`* declares the synced fields; `Camera` adds behaviour on top). It is a SpiriSynq `Position` in particular: `relative_to` and `mount` say what it is mounted on and how, and `pose` is where it is and which way it looks, which is what lets an overlay draw on things in the world (*Drawing on things in the world*). Two things about it are easy to get wrong:

Nothing is read-only. Fields like `max_supported_width` or `received_width` are described as "what the device reported" or "what was actually received", but they are ordinary writable fields — a remote peer can already set any of them over SpiriSynq, so a property setter or a one-way binding would only hide that, not enforce it. This is why `ui.py` binds every field two-way with `bind_value` rather than `bind_value_from`:

being able to type a wrong value in and watch what downstream does with it is a deliberate debugging feature, not an oversight. The only fields with a one-way binding in the debug UI are ones with no meaningful counterpart to write back to — the bandwidth meter and the read-out frame tags (see below).

Authoritative vs. mirror. A `Camera` you construct normally (`Camera("/dev/video0")`) is *authoritative*: it actually owns the device, and SpiriSynq prefixes its topic with this machine's hostname and exposes its `@remote_method` RPCs. A `Camera` rebuilt from a SpiriSynq rehydrate reply (`Camera.from_topic(...)` on a remote peer) is a *mirror*: same synced fields, but it never resolves `source_str` into a local handler, because a device path on the mirror's machine may mean something completely different (or nothing at all) locally. A mirror therefore always starts with `handler is None` and cannot `start()` — it exists to observe, not to capture. This is controlled by the `synq_authoritive` constructor argument, which defaults to the right answer for both cases; pass it explicitly only if you're doing something unusual (see `ui.py`'s `get_camera()` for an example of spelling it out for clarity even though the default already does the right thing).

Frame-derived fields never travel as their own field. `received_width`, `received_height`, `received_ratio`, and the EXIF-derived `exif_tags`/`exif_timestamp` all describe one specific frame. SpiriSynq syncs each field independently with no ordering guarantee between two fields, so publishing "this frame is 1920x1080" next to "here is the image" is a race — a peer can read the width against a *different* image than the one it names. Instead these values are written into the JPEG's EXIF by *`SpiriCamera.exif`* and read back out of `cam.image` by `Camera._on_image_changed`, identically on the authoritative node and on every mirror. If you add a new field that describes "the current frame" rather than "the camera's settings", it almost certainly belongs in EXIF, not as a new synced attribute — copy the pattern of `received_width` rather than adding a sibling to `max_width`.

4.3 Adding a new source handler

A handler is a `SourceBase` subclass that gets discovered automatically by being imported — there's no registry to edit. Concretely, to add one:

1. Create `src/SpiriCamera/sources/my_protocol.py`.
2. Subclass *`SourceBase`* (or *`OpenCVSource`* if `cv2.VideoCapture` can already speak your protocol — this is what `v4l.py` and `network.py` do; only write against the raw `SourceBase` contract if you need something OpenCV can't do, the way `testimage.py` renders SVGs itself).
3. Set `schemes = ("myproto",)` and implement `open`, `close`, `read`, and `is_open`. Override `handles()` too if you need to claim schemeless strings the way `v4l.py` claims bare device paths.
4. Import the module from `src/SpiriCamera/sources/__init__.py`'s `# Importing these modules is what registers their handlers` block.

Minimal skeleton, modeled on the real handlers:

```
"""My protocol camera source."""

from __future__ import annotations

import numpy as np

from SpiriCamera.sources.base import (
```

(continues on next page)

(continued from previous page)

```
    CaptureSettings,
    SourceBase,
    SourceCapabilities,
    SourceError,
    SourceURL,
)


class MyProtocolSource(SourceBase):
    """Reads frames from my-protocol://<target>."""

    schemes = ("myproto",)

    def __init__(self, url: SourceURL) -> None:
        super().__init__(url)
        self._device = None

    @property
    def is_open(self) -> bool:
        return self._device is not None

    def open(self, settings: CaptureSettings) -> SourceCapabilities:
        try:
            self._device = _connect(self.target, settings)
        except OSError as exc:
            raise SourceError(f"could not open {self.target!r}: {exc}") from␣
↪exc
        return SourceCapabilities(
            max_supported_width=self._device.width,
            max_supported_height=self._device.height,
        )

    def close(self) -> None:
        if self._device is not None:
            self._device.release()
            self._device = None

    def read(self, settings: CaptureSettings) -> np.ndarray | None:
        # Honour `settings` on every call, not just at open() -- this is
        # what lets a live resolution/framerate change take effect
        # without restarting the camera.
        return self._device.grab_frame(settings)
```

A few contract details that are easy to miss because they only show up as bugs in a UI far from the handler:

- **`read()` may return `None`** when there's no new frame yet (e.g. a network stream that hasn't delivered one). `Camera.read()` then re-serves the previous `cam.image` rather than raising, so returning `None`

is the normal way to say "nothing changed", not an error.

- **Return the *same* array object** for an unchanged frame (a static test pattern, a paused file) rather than an equal-but-distinct copy. `Camera._encode` caches by identity (`is`, not `==`) specifically so a source that isn't producing new pixels doesn't pay for re-encoding every capture tick.
- **`open()` on an already-open source must be a no-op** that just re-reports capabilities — `Camera.start()` on an already-running camera doesn't call `open()` at all, but a source can still see a redundant `open()` from other paths and must not double-acquire the device.
- **Never raise from resolution** (`handles`/`from_url` aside from a genuinely malformed target) — an unresolvable or not-yet-openable source should surface through `SourceError` at `open()`/`read()` time, because `Camera` is designed to be constructed from a half-typed, live-edited source string without raising. If your handler needs validation, do it in `open()`, not in the constructor.

4.4 The overlay layer

`SpiriCamera.overlay` bakes SVG "HUD" widgets into a frame before it is encoded. A widget (*`HudWidget`*) is its own `SyncableObject`, published wherever its author runs (a UI panel, an ML detector) — a camera never creates one, it only mirrors the widgets named in its `overlay_widgets` field (*`OverlayMixin`*, mixed into *`CameraBase`*). This split — author owns the widget, camera only opts in — is what lets several cameras share one widget definition while each binds it to its own data and placement.

A widget's SVG is templated with MiniJinja against three separate namespaces (plus `view` and `item` for a widget placed on the world; see below), and *why there are three* is the one thing worth understanding before touching this code:

- `exif` — the current frame's tags.
- `frame` — the current frame's actual, as-received width/height/ framerate and a render-time timestamp. These are injected directly rather than read through `objects` because they are exactly the fields the "Frame-derived fields never travel as their own field" rule above forbids publishing as synced attributes — there is no topic a widget could declare to reach them.
- `objects` — the latest field values of any other SpiriSynq object a widget declares for itself, by alias, via a `{# object: <alias> = <default topic> #}` comment next to where the alias is used. The alias exists because a real topic can contain characters (a hyphen in a hostname) that are not valid inside a `{{ }}` expression, and because a camera rendering the widget can rebind an alias to a different concrete object without touching the shared widget definition (see `overlay_widgets`' `"bindings"` key).

`SpiriCamera.overlay.resolve_objects` cross-checks MiniJinja's own static analysis of what a template *reads* against what it *declares*, and raises immediately if an alias is used with no matching declaration — a widget author gets that error at authoring time rather than a silently blank value baked into every frame.

Rendering never scales. `rasterize()` renders each widget at its own natural pixel size (`Picture.get_size()`); `composite()` alpha blends it onto the frame at whatever position `anchor_position()` computes and clips the rest. A widget author who writes `font-size="24"` gets a real 24px regardless of the camera's resolution — scaling the raster to fit a declared box was rejected because it either blurs small text or makes it illegible on a 4K frame. Text is drawn by `thorvg`, using the bundled `Miracode` font registered once at import time; `thorvg` never falls back to an OS-installed font, so a widget that references an unloaded font family renders nothing for that text, silently.

Failure is per-widget, per-frame, and never fatal. A widget that fails to render — a template error, a declared object with no value yet — is skipped for that frame and logged once (`OverlayMixin._overlay_complain` de-duplicates by message so a persistently-broken widget does not spam the log every frame). One broken overlay is not allowed to stop the camera from publishing.

Mirroring is lazy and cheap to re-check. `_overlay_sync_widgets` and `_overlay_sync_objects` run once per frame but only do network work (mirroring a widget or a data-source object) the first time a topic is seen; anything already mirrored is trusted to update itself live via its own SpiriSynq subscription. That is what makes it safe to call every frame, which is how newly-added or newly-rebound `overlay_widgets` entries get picked up promptly without wiring fine-grained `EventedDict` change signals.

Placing a widget on the world is opt-in, and the template never does the geometry. A widget that declares `{# anchor: ... #}` or `{# project: ... #}` (`world_placement()`) is drawn by where its objects are in the frame tree: `_overlay_renders()` projects their poses through the camera's own pose and lens (*`SpiriCamera.projection`*), and either centres a copy of the widget on each, or hands a frame-wide widget the pixels as `view`. That is why a camera is a `Position`. A relay camera follows its upstream's pose and lens through the source contract (`SourceCapabilities.frame`, `SourceBase.frame_intrinsics()`), so a source never reaches back into the camera. Both render paths, burned in and client-side, go through `_overlay_renders()` and `_overlay_position()`, so they place widgets the same way. The guide for widget authors is *Drawing on things in the world*.

See the *`SpiriCamera.overlay`* module docstring for the full reasoning, and *`camera_metrics_widget()`* for a worked example of a ready-made widget that reads both `frame` and its own camera's live fields via `objects`.

4.5 The debug UI's bandwidth and latency readout

`SpiriCamera.ui` runs one `Camera` per process (`get_camera()`, module global) and serves its frames over a plain HTTP route (`GET /camera/frame`) rather than pushing them over the websocket as base64 — `ui.image`/`QImg` cross-fades on every source change, which at video framerate looks like a permanent dim pulse, and pushing to every client regardless of whether it can keep up defeats the point of a pull-based readout. Read the module docstring in *`SpiriCamera.ui`* for the full reasoning; the parts worth knowing if you're debugging the numbers it shows:

- **fps / KiB/s / kb/s** come from `RateMeter`, which is fed only when `serve_frame()` sees a *new* frame object (compared by identity, via `_last_counted`) — a stopped camera being polled repeatedly reports 0 fps rather than the rate of the polling. `_last_counted_lock` exists because two browser tabs can both read the stale value before either writes the new one, which without the lock double-counts a single frame across two concurrent requests.
- **Frame age** is *not* "now minus this frame's timestamp" sampled once — that number saws between zero and a full frame interval depending on when exactly you read it, and says more about polling phase than actual latency. `RateMeter.age()` instead reports `max(mean_age_over_window, true_age_of_newest_frame_seen)`: the mean is the stable number while frames are flowing, and the true age is what it decays into once they stop — a camera that died five minutes ago should read "5 min old", not fall back to a rolling average of zero.
- **Clock skew**: age is computed from the frame's own EXIF capture timestamp against `time.time()` on the machine serving the debug UI, clamped at zero. Against a remote/mirrored camera this is only as good as the agreement between the two machines' clocks — an age of "0 ms" on a badly-skewed pair

doesn't mean zero latency, it means the remote clock is ahead.

4.6 WebRTC egress, ingest, and WHEP

`SpiriCamera.webrtc` is where every bit of aiortc lives; nothing else in the package imports aiortc/FastAPI at module scope except the two signaling doors that use it:

- *`Camera.webrtc_offer`* — a single `@remote_method`, reachable over zenoh like any other RPC.
- *`SpiriCamera.whep`* — a standalone FastAPI app implementing WHEP (`POST /whep`, `DELETE /whep/{session_id}`), independent of `ui.py`'s NiceGUI app, run via `SpiriCamera whep-serve`.

Both call the same `SpiriCamera.webrtc._negotiate`, so a viewer that arrives over zenoh and one that arrives over HTTP land in the same session registry and get the same video track and tags channel — the signaling transport is the only thing that differs.

The shared background event loop. An `RTCPeerConnection` has to keep running after the call that created it returns, but the zenoh RPC path calls into `webrtc_offer` synchronously from a queryable callback thread with no event loop at all. `SpiriCamera.webrtc.get_loop()` starts one daemon thread running its own loop forever, the first time anything asks for it, and every peer connection this module creates lives there for good. `negotiate_sync` blocks the calling thread for a result; `negotiate_async` is the same call awaited from a caller that already has a loop (uvicorn's, in `whep.py`).

Tags only travel one way. A single SDP answer can only describe `m=` sections the offer already listed — an answerer cannot add a data channel that was not in the offer without a second negotiation round. So the *viewer* has to create the data channel (labelled `spiricamera-tags`) before sending its offer; `SpiriCamera.webrtc` listens for it arriving via the `"datachannel"` event and, if it never does, simply sends video with no tags. This means a generic WHEP player (a browser, `ffplay`) gets video only; only a SpiriCamera-aware client gets the per-frame `{"timestamp": ..., "tags": {...}}` JSON, read straight out of that frame's EXIF via `SpiriCamera.exif.extract` — the same tags already embedded per the "Frame-derived fields never travel as their own field" rule above, not a second copy of them.

Ingest is a source handler. `SpiriCamera.sources.webrtc.WHEPSource` is the mirror image: a `whep+http://host/path` or `whep+https://...` source string (compound schemes, since `NetworkSource` already claims bare `http`/`https`) pulls a remote WHEP publisher's stream in as an ordinary `SourceBase`, negotiating on `open()` and reading decoded frames off a background task on the same shared loop.

Another camera is a source too. `SpiriCamera.sources.spirisynq.SpiriSynqSource` (`spirisynq://<topic>`) mirrors an upstream camera on `open()` and returns its latest frame, decoded and fitted to the requested bounds. It never imports `camera.py`: it looks the upstream's type tag up with `Session.list_topics` and mirrors it with whatever class the session has registered under it (the real `Camera`, in any process running one), falling back to `from_topic_untyped` only when there is none. The order matters — `from_topic_untyped` re-registers the tag with a synthesized class, which breaks every later `Camera.from_topic` on that session. `Camera.start()` opens its handler under `synq_session.as_default()`, so the mirror lands on the camera's own session even when a remote `running = True` triggers the start from a zenoh thread. Upstream frame tags come through `SourceBase.frame_tags()`, merged under the camera's own.

Sources that are not there yet, or any more. A background camera treats a `SourceError` from `open()` as "not yet": it stays `running`, reports `waiting for source: <reason>`, and the capture loop retries the open every `_SOURCE_RETRY_INTERVAL`. The loop reopens any handler whose `is_open` goes false, so

the contract for a source that loses its device mid-stream is simply to close itself and raise from `read()`. `V4LSource` does that when a failed read finds its device node gone (a failed read with the node present is only a dropped frame). `SpiriSynqSource` does it on the upstream's tombstone (a clean shutdown), or — since a crash sends none — when no frame has arrived for `STALE_AFTER` seconds and the topic no longer answers `list_topics`, or answers from a different process. It uses `list_topics` rather than an `sr_metadata` RPC for this because an absent topic is silently no reply there, where the RPC logs an error on every retry.

4.6.1 Hooking a `whep-serve` camera up to MediaMTX

MediaMTX can pull a WHEP stream directly as a path source — no WHIP push, no extra code on our side. Point a path at whatever `SpiriCamera whep-serve` is listening on:

```
# mediamtx.yml
paths:
  spiricamera:
    source: whep://127.0.0.1:8889/whep
```

MediaMTX acts as the WHEP *client* here (the same role our own `SpiriCamera.sources.webrtc.WHEPSource` plays), and re-serves the video to as many RTSP/HLS/WebRTC/etc. viewers as you like from that one path.

The `spiricamera-tags` data channel does not survive this hop. MediaMTX's WHEP-pull offer only asks for a recvonly video track — no data channel — so per the one-way rule above (offer decides which `m=` sections can exist), our answer never includes one, and there is nothing to carry the tags through. This isn't a bug to work around: as of the current release, MediaMTX's WebRTC implementation has no data channel support at all (checked its full config reference for `datachannel`/`metadata`/`sei` — nothing), so no WHEP offer it sends will ever open one. A viewer that needs tags has to talk to `whep-serve` directly, bypassing MediaMTX, so its own offer can open the channel.

The one metadata path MediaMTX *does* carry end-to-end is KLV — it implements RFC 6597 (`SMPTE336M/90000`) as a first-class RTSP track type, cross-muxing between MPEG-TS/SRT ingest and RTSP output. That would mean encoding our tags as MISB ST 0601 KLV and giving SpiriCamera an RTSP/SRT *output* carrying a synchronous KLV track — unrelated to WHEP/WebRTC entirely, and not implemented yet (see `todo.md`).

4.7 Where things are tested

`tests/` exercises the source handlers and `Camera` against real `SpiriSynq` peers over a namespaced zenoh session (see recent history around "namespace the test suite's zenoh peer" for why — tests must not collide with a zenoh session already running on the same machine). Run the full suite with:

```
python -m pytest
python -m pytest --cov=SpiriCamera
```

When adding a new source handler, add a corresponding test module alongside the existing ones rather than folding it into `test_camera.py` — handler tests exercise `open`/`close`/`read` in isolation from the capture thread, which is what makes it possible to test protocol edge cases without racing a background thread.

CHAPTER

FIVE

TUTORIAL: YOLO DETECTION BOXES AS A LIVE OVERLAY

This walks through `examples/yolo_overlay.py`, a standalone script that:

1. Attaches to an *existing* camera over SpiriSynq (a mirror — it never opens a device itself).
2. Runs YOLO object detection on each new frame.
3. Publishes the detected boxes as their own SpiriSynq object.
4. Publishes a `HudWidget` that reads that object and draws the boxes as SVG rectangles, and attaches it to the camera so they show up composited on its picture.

It's a separate process from the camera and from any UI — run it next to whatever is already serving the camera, point it at that camera's topic, and stop it when you're done; nothing about the camera or the overlay system needs to know it exists beforehand.

Install the one extra dependency this needs beyond SpiriCamera's own, via the `examples` extra:

```
uv sync --extra examples
```

(or `pip install ultralytics` directly, if you're not using `uv`)

Then run it against a camera another process already has open:

```
python examples/yolo_overlay.py some-host/dev-video0
```

5.1 Running it against the NiceGUI debug UI

The debug UI (`uv run python -m SpiriCamera.ui`) is just another process that owns or mirrors a camera — the detector script attaches to *that* camera's topic exactly like it would any other. The one piece of information you need first is the topic, and the easiest way to get it is to ask SpiriSynq the same question the UI's own "Cameras on the network" panel does:

```
python -c "
from SpiriSynq.session import current_session
for meta in current_session.get().list_topics(type_filter='Camera'):
    print(meta['topic'])
"
```

This prints every camera currently advertising itself, including the UI's own — e.g. `$(hostname)/spiricamera_testimage` for the default `testimage://` source (the topic's first segment is whatever machine it's running on). (The "Synq Topic" field shown in the UI's Device card is only the *local* half of this — `spiricamera_testimage` without the hostname — so prefer the discovery one-liner over reading it off the page.) If the UI is itself mirroring another camera rather than running a device, this lists that camera's topic instead — either way, whatever the UI is currently showing frames from is what gets discovered here.

Then run the detector against it:

```
uv sync --extra examples   # once, if you haven't already
python examples/yolo_overlay.py $(hostname)/spiricamera_testimage
```

Refresh the UI's page (or hit the refresh icon on its overlay panel) and the detection boxes should appear composited on the live picture within a frame or two — no restart of the UI needed, since attaching the overlay is done by editing `overlay_widgets`, a synced field the camera picks up live.

Before the detector is running, the UI just shows the raw picture:



Once `yolo_overlay.py` is attached, green boxes appear around each detection, composited directly into the frame the UI is already displaying:



5.1.1 Seeing an actual box

The default `testimage://` pattern is a color-bar test card — YOLO correctly finds nothing in it, so `detections.boxes` will just stay empty and no rectangles will appear. To see a real detection without plugging in hardware, point the UI's "Source string" field (or `SpiriCamera run testimage://yolo_demo` on the CLI) at `testimage://yolo_demo` instead: a real photo (borrowed from `ultralytics`'s own sample assets, so it's only available once the `examples` extra is installed) panned slowly across frame, with people and a bus for YOLO to actually find.

5.2 How it fits together

5.2.1 Getting frames without owning the device

```
from SpiriCamera import Camera

cam = Camera.from_topic(args.topic)
```

This is the mirror pattern described in *Getting Started* — no device is opened locally, `cam.image` is kept up to date in the background, and there is no `read()` to drive. The script runs a tight loop, not a poll: there's no sleep pacing it, it just runs detection back-to-back as fast as `model.predict` allows, skipping a cycle only when `cam.image` is the exact same object it already processed:

```
last_jpeg = None
while True:
    jpeg = cam.image
    if not jpeg or jpeg is last_jpeg:
        continue
    last_jpeg = jpeg
    ...
```

In practice a real source produces frames faster than YOLO can keep up, so that identity check almost never fires — it only matters at startup (before a first frame has arrived) or against a very slow source. This is deliberately *not* event-driven: an earlier version connected to `cam.events.image` and ran detection from that callback, but `cam.events.image` fires on zenoh's own callback thread, and psygnal holds a lock for a signal's entire emission — for as long as a connected slot is running. Detection taking a few hundred milliseconds meant `cam.events.image.disconnect()`, called from the main thread during shutdown, blocked on that same lock until inference finished, and the non-daemon zenoh thread underneath it couldn't be joined until the callback returned either — the process wouldn't die on `Ctrl-C`. The plain loop above sidesteps all of that: there's no signal connection to disconnect and no callback thread to wait on, so `Ctrl-C` stops it immediately.

5.2.2 A note on overlays feeding back into detection

Attaching the widget to the mirrored `cam` (`cam.overlay_widgets[...] = {}`) bakes the boxes directly into `cam.image` server-side — see *`read()`*, which composites overlays before encoding unless *`overlay_client_render`* is set. That means this same script's *next* detection pass runs on a frame that already has its own boxes drawn on it: the overlay it just added is fed straight back into the thing deciding what to add. Harmless here — YOLO doesn't mistake a green rectangle for a person or a bus, so it doesn't compound — but it's not the shape a real deployment wants. There, the node that owns the camera and the node

that runs detection and publishes the overlay are two separate processes, and the camera-owning one sets `overlay_client_render = True` so overlays are only ever composited for display (in a browser, via *`render_overlays_for_client()`*) and never baked into the frames anything downstream analyzes.

5.2.3 Publishing detections as a generic SpiriSynq object

The overlay system's data sources aren't camera-specific — *any* `SyncableObject` with synced fields can be one (see *`SpiriCamera.overlay`*'s module docstring for the mechanism, and `docs/architecture.md` for why it's built this way). So the detector defines the smallest object that fits:

```
@dataclass
class Detections(SyncableObject):
    boxes: list = field(default_factory=list)


detections = Detections(synq_topic="yolo-detector/detections",␣
↪synq_authoritive=True)
...
detections.boxes = boxes  # reassigning the field publishes it
```

Each entry in `boxes` is a plain dict — `{"x", "y", "w", "h", "label", "score"}` in frame pixel coordinates. There's no dedicated "list of boxes" field type in SpiriSynq; a plain `list` field works because assigning it wholesale is just a normal synced-field change like any other.

5.2.4 Authoring the overlay widget

A `HudWidget`'s SVG is a MiniJinja template. It declares the object it needs with a comment (stripped before rendering) and reads its fields through the alias that comment names:

```
{# object: det = yolo-detector/detections #}
<svg ... viewBox="0 0 {{ frame.width }} {{ frame.height }}">
  {% for box in objects.det.boxes %}
  <rect x="{{ box.x }}" y="{{ box.y }}" width="{{ box.w }}" height="{{ box.h }}
↪"
        fill="none" stroke="lime" stroke-width="3"/>
  {% endfor %}
</svg>
```

There's no built-in "bounding box" widget type — overlays are always hand-authored SVG, so looping over `objects.det.boxes` and drawing `<rect>` elements *is* the box-drawing widget, not a shortcut around one.

5.2.5 Colouring boxes by class

The template above draws every box the same colour. Telling classes apart at a glance – people in one colour, vehicles in another – needs nothing from Python or SpiriSynq, only more of the MiniJinja already in the template: a `{% set %}` dict literal and a lookup inside the loop. This is the step actually shipped in `examples/yolo_overlay.py`; it's broken out here because it's a good worked example of MiniJinja doing real logic, not just substitution:

```
{# object: det = yolo-detector/detections #}
{% set colors = {"person": "lime", "bus": "orange", "car": "deepskyblue"} %}
<svg ... viewBox="0 0 {{ frame.width }} {{ frame.height }}">
  {% for box in objects.det.boxes %}
  {% set color = colors[box.label] | default("yellow") %}
  <rect x="{{ box.x }}" y="{{ box.y }}" width="{{ box.w }}" height="{{ box.h }}
↪"
        fill="none" stroke="{{ color }}" stroke-width="3"/>
  {% endfor %}
</svg>
```

Two things worth noticing:

- `colors` is a plain template-local mapping, not a second declared object – there's nothing about a class-to-colour scheme that's live or worth publishing over SpiriSynq, so it's just data baked into the template alongside the markup, exactly like a constant in any other language.
- `colors[box.label]` indexes that mapping with each box's own `label` field. Since *`SpiriCamera.overlay`* renders templates with MiniJinja's strict undefined behavior (see its module docstring), a `label` the map doesn't have an entry for would otherwise fail the whole widget's render for *every* box, not just the unrecognised one – one dropped bounding box shouldn't blank out the rest. The `| default("yellow")` filter is what avoids that: it only replaces the specific lookup that came back undefined, falling back to a colour for any class the map doesn't name.

5.2.6 Making the label readable on any background

A coloured label drawn straight onto the picture disappears the moment it lands on a similarly-coloured or bright patch of frame – a lime label over grass, say. The usual SVG fix is an outline: give the `<text>` a `stroke` and set `paint-order="stroke"` so it's drawn under the fill. That doesn't work here – thorvg's SVG parser silently ignores `stroke`, `stroke-width`, and `paint-order` on `<text>` altogether (the same family of parser gap *`SpiriCamera.overlay`* already documents for `dy`, see its module docstring). `fill` on `<text>` does work, though, which is enough to fake an outline by drawing the label several times, nudged a pixel in each diagonal direction in black, then once more on top in the real colour:

```
{% set halo_offsets = [[-1,-1],[1,-1],[-1,1],[1,1]] %}
...
{% for dx, dy in halo_offsets %}
<text x="{{ label_x + dx }}" y="{{ label_y + dy }}" fill="black"␣
↪font-size="16">
  {{ box.label }} {{ box.score }}
</text>
{% endfor %}
<text x="{{ label_x }}" y="{{ label_y }}" fill="{{ color }}" font-size="16">
  {{ box.label }} {{ box.score }}
</text>
```

`halo_offsets` is a list of two-element lists, unpacked with `{% for dx, dy in halo_offsets %}` exactly like Python would. Deliberately not a background chip behind the text (a filled or translucent `<rect>`, the way *`camera_metrics_widget()`* does it) – a halo keeps the label sitting directly on the picture with

nothing else added to it, at the cost of five text draws per box instead of one. Either is a legitimate choice; which one reads better is a matter of taste for a given overlay, not a limitation of one over the other.

The widget is published the same way the built-in *tiger_widget()* example is, just from an external process instead of from inside SpiriCamera:

```
widget = HudWidget(
    synq_topic="yolo-detector/boxes_overlay",
    synq_authoritive=True,
    anchor="top_left",
    svg_template=svg_template,
)
cam.overlay_widgets[widget.synq_absolute_path] = {}
```

That last line is what makes the mirrored cam actually render it — overlay_widgets is a synced field itself, so this takes effect on the camera's very next frame, wherever that camera's process is running, no restart needed. Note the key is widget.synq_absolute_path, not the bare synq_topic passed to the constructor — the widget's *base* topic prefix (e.g. a session's default namespace) is folded in, and that's the value the camera looks up by.

5.3 How many lines is this, really

The whole script is about 60 lines of actual control flow (imports, argument parsing, the detection loop, cleanup) plus a roughly 15-line SVG template as a string constant — call it **~75 lines** for a from-scratch, independently-runnable detector. None of it is boilerplate specific to this tutorial: the same shape — mirror a camera, define a small SyncableObject, author a template that reads it — is how any external process publishes any kind of overlay data, not just bounding boxes.

5.4 Cleaning up

The script publishes two objects (detections and widget) and closes both, plus the mirrored cam, on Ctrl-C. Nothing it created outlives the process — the camera stops rendering the overlay the moment the HudWidget it was reading disappears from SpiriSynq.

CHAPTER

SIX

API REFERENCE

The SpiriCamera package provides camera functionality for video capture and frame processing, built into the SpiriSynq ecosystem.

6.1 Classes

SpiriCamera — Work with Cameras in SpiriSynq.

class SpiriCamera.**Camera**(*source=''*, ** (Keyword-only parameters separator (PEP 3102))*, *quality=80*, *max_width=1920*, *max_height=1080*, *max_framerate=30*, *mimetype='image/jpeg'*, *synq_topic=''*, *synq_auto_start=True*, *synq_authoritive=None*, ***rehydrated*)

Bases: *CameraBase*

A live camera: resolves a source, captures frames, publishes them.

Usage:

```
cam = Camera("/dev/video0", quality=85)
cam.start()                 # begins the background capture thread
frame_bytes = cam.image     # whatever it has captured so far
cam.stop()
```

or as a context manager, which starts on entry and stops on exit:

```
with Camera("testimage://", max_width=1280, max_height=720) as cam:
    frame_bytes = cam.image
```

Treat a running `Camera` like a small daemon, not a handle to poll by calling `read()` yourself: leave `start()` on its default `background=True` and let the capture thread run its own loop, paced to `max_framerate`. To change the rate, set `max_framerate` — it is live and takes effect on the next frame — rather than calling `read()` less often. `start(background=False)` exists for scripted, frame-at-a-time use (tests, one-off captures); it is not the pattern for anything long-running.

This is the *authoritative* role: actually owning and driving a device. Code that merely wants another process's frames should not construct a second `Camera` this way — it should mirror the existing one with `from_topic()` and either poll `image` or connect to its psygnal event (`cam.events.image.connect(...)`); see the "Consuming an existing camera" section of the getting-started guide.

While running, a background thread captures at `max_framerate` and assigns each encoded frame to `image`, which publishes it to SpiriSynq. Settings are live: changing `max_width`, `quality`, or `max_framerate` — locally or from a remote peer — takes effect on the next frame without a restart.

An unusable source is not fatal. Construction never raises for a bad source string; the reason lands in `camera.source.error` and `start()` is what refuses. This keeps the camera bindable to a live-edited input field.

Parameters

- **source** (`SourceInfo`)
- **quality** (*int*)
- **max_width** (*int*)
- **max_height** (*int*)
- **max_framerate** (*int*)
- **mimetype** (*str*)
- **synq_topic** (*str*)
- **synq_auto_start** (*bool*)
- **synq_authoritive** (*bool*)
- **rehydrated** (*object*)

property handler: ***SourceBase*** **| None**

The resolved source handler, or `None` if unresolvable.

start(*, *background=True*)

Open the source and begin capturing.

Starting an already-running camera is a no-op.

6.1.1 Parameters

background

[bool] Whether to run a capture thread. With `False` the camera opens but captures nothing until the caller drives `read()` — useful for scripted, frame-at-a-time use.

A source that resolves but will not open yet – a device that is not plugged in, a `spirisynq://` topic nobody publishes yet – does not stop a background camera from starting. It starts *waiting*: `running` is true, `status` reads `"waiting for source: <reason>"`, and the capture thread keeps trying to open the source until it works. The same happens if the source is lost while running.

6.1.2 Raises

CameraError

If the source string could not be resolved, or – only with `background=False`, where no thread is left to wait – the source refused to open.

Parameters
background (*bool*)

Return type
None

stop()

Stop capturing and release the source.

Stopping an already-stopped camera is a no-op. Safe to call from the capture thread itself, in which case the thread is not joined.

Return type
None

close()

Stop the camera and release its SpiriSynq resources.

Return type
None

webrtc_offer(*sdp*, *type='offer'*)

Negotiate a WebRTC session publishing this camera's frames.

The zenoh-native counterpart to *SpiriCamera.whep*'s HTTP route: both end up in *SpiriCamera.webrtc.negotiate_sync()*, so a viewer reached either way gets the same video track and tags channel.

6.1.3 Parameters

sdp
[str] The remote peer's SDP offer.

type
[str] The offer's SDP type, almost always `"offer"`.

6.1.4 Returns

dict[str, str]
`{"session_id": ..., "sdp": ..., "type": "answer"}`.

Parameters

- **sdp** (*str*)
- **type** (*str*)

Return type
dict[str, str]

set_intrinsics(*origin*, *width=0*, *height=0*, *fx=0.0*, *fy=0.0*, *cx=0.0*, *cy=0.0*, *hfov=0.0*, *vfov=0.0*)

Report this camera's lens, as *origin* knows it.

Either a full pinhole model – `width`, `height`, `fx` and `fy`, with `cx`/`cy` defaulting to the image centre – or just a field of view in degrees, `hfov` and optionally `vfov`, which becomes a centred model at the camera's own frame size once it knows one. A later report from the same origin replaces this one; `intrinsics` is whichever origin ranks highest in `INTRINSICS_ORIGINS`.

Plain numbers rather than an `Intrinsics`, so a peer that doesn't have this package (SpiriMav) can call it.

6.1.5 Raises

ValueError
For an unknown origin, or numbers that aren't a lens.

Parameters

- **origin** (*str*)
- **width** (*int*)
- **height** (*int*)
- **fx** (*float*)
- **fy** (*float*)
- **cx** (*float*)
- **cy** (*float*)
- **hfov** (*float*)
- **vfov** (*float*)

Return type
None

clear_intrinsics(*origin*)

Forget what *origin* reported, falling back to the next one.

Parameters
origin (*str*)

Return type
None

capture_settings()

Snapshot the currently requested capture settings.

Values arriving from UI bindings or remote peers may be floats or `None`; they are coerced to positive integers here so sources can trust what they are given.

6.1.6 Returns

CaptureSettings
The settings to hand to the source.

Return type
CaptureSettings

describe_capabilities()

Summarise what the source reported, for logs and the CLI.

6.1.7 Returns

str
Something like `1920x1080 @ 30fps`. Sources that render at whatever size they are asked for report no fixed limits, and are described as `any size` / `any rate`.

Return type
str

read_frame()

Grab one raw frame from the source.

6.1.8 Returns

np.ndarray | None
A BGR frame, or `None` if the source had nothing new.

6.1.9 Raises

CameraNotStartedError
If the camera is not running.

SourceError
If the source failed unrecoverably.

Return type
ndarray | None

read()

Capture, encode, and publish a single frame.

The encoded frame is tagged, then assigned to `image`, which is what publishes it to SpiriSynq. Assigning is also what fills in `received_width` and the rest, since those are read back out of the frame rather than measured here.

6.1.10 Returns

bytes
The encoded frame, in `mimetype` format. If the source had no new frame, the previous one is returned unchanged.

6.1.11 Raises

CameraNotStartedError
If the camera is not running.

FrameUnavailableError
If the source produced nothing and no earlier frame exists.

Return type
bytes

property exif_tags: dict[str, str]

The tags on the current `image`.

Read back out of the frame itself rather than stored, so this is never out of step with what is actually on screen — on the authoritative node or on any mirror, since both learn it the same way, from `_on_image_changed()`.

property exif_timestamp: float

Unix timestamp the current `image` was captured at, or `0.0` if it carried none.

Set by whichever node took the frame, so comparing it against a local clock is only as good as the agreement between the two.

exif_set_tags(*provider*, *tags*)

Replace one provider's tags, leaving every other provider's.

This is the primitive for more than one piece of software tagging the same camera's frames without stepping on each other: each provider owns its own bucket, keyed by `provider`, and setting one never touches another's. Call it again with the same `provider` to replace what it set before.

6.1.12 Parameters

provider
[str] A name identifying who is setting these tags — your module or component name is a reasonable choice. Two providers using two different names can never clobber one another, no matter what tags they choose or when they call this.

tags
[Mapping[str, str]] The complete set of tags this provider wants embedded in every frame from now on. An empty mapping is the same as `exif_clear_tags()`. Unlike `exif_update()`, a value of `""` is kept rather than dropped, which lets a provider that genuinely needs to blank out a built-in tag — omitting `source` from a frame headed somewhere public, say — do exactly that.

Parameters

- **provider** (*str*)
- **tags** (*Mapping[str, str]*)

Return type
None

exif_clear_tags(*provider*)

Remove everything `provider` has set, and nothing else.

6.1.13 Parameters

provider
[str] The same name passed to `exif_set_tags()`. Clearing a provider that never set anything is not an error.

Parameters
provider (*str*)

Return type
None

exif_tag_providers()

Names of the providers currently contributing custom tags.

6.1.14 Returns

frozenset[str]
Provider names with at least one tag set right now. Does not include the built-in tags `exif_tags_for_frame()` adds on its own, which have no provider name.

Return type
frozenset[str]

exif_update(***tags*)

Merge tags into the default, unscoped tag bucket.

A convenience for the common case of one caller adding a tag or two without needing a provider name of its own. Two *different* callers doing this concurrently still share this one bucket by construction and can still overwrite each other; once that matters, give each caller its own name and use `exif_set_tags()` instead.

6.1.15 Parameters

****tags**
[str] Tags to add or replace in the default bucket. A value of `""` drops that one tag from the default bucket, leaving the rest of it, and every other provider, untouched.

Parameters
tags (*str*)

Return type
None

exif_tags_for_frame(*frame*)

Build the tags to write into the next encoded frame.

The override point for tagging. A subclass with a GPS fix or a gimbal angle to attach on every frame should extend what this returns; a caller or another piece of software adding metadata at runtime should use `exif_set_tags()` / `exif_update()` instead, which this merges in afterwards.

Tags come in three layers, each overriding the one before: tags the source says arrived with the frame (*`frame_tags()`*, e.g. an upstream camera's GPS tags through a `spirisynq://` source), then this camera's built-ins, then providers. The built-ins describe the node that produced *these* bytes, so they replace an upstream's `source` and `topic` – except the capture time, which is kept from the inherited tags when they carry one.

Providers are merged in over the built-ins in sorted-name order, so the result does not depend on which provider happened to register first — but two providers naming the *same* tag will still resolve in that fixed order regardless, which is worth avoiding by prefixing tag names with something distinguishing rather than relying on.

Dimensions are deliberately absent: the JPEG header already states them, and `_on_image_changed()` reads them from there. A tag saying something the container also says is a tag that can disagree with it.

6.1.16 Parameters

frame
[np.ndarray] The raw BGR frame about to be encoded.

6.1.17 Returns

dict[str, str]
Tags to embed. Empty means embed nothing.

Parameters
frame (*ndarray*)

Return type
dict[str, str]

class SpiriCamera.**CameraBase**(*synq_topic*, *synq_base_topic=None*, *synq_session=<factory>*, *synq_authoritive=False*, *synq_lazy_publish=False*, *synq_publish=True*, *synq_receive=True*, *synq_auto_start=True*, *synq_check_receive_types=True*, *synq_auto_rehydrate_on_missing_parent_timeout=5.0*, *synq_mtime=-1*, *synq_is_deleted=False*, *_synq_callbacks=<factory>*, *_synq_last_rehydrate_time=0.0*, *relative_to='unknown'*, *mount=None*, *pose=None*, *_parent_subscriber=None*, *_parent_pose=None*, *_parent_live=False*, *_parent_generation=0*, *overlay_widgets=<factory>*, *overlay_client_render=False*, *source_str=''*, *source=<factory>*, *vendor=''*, *model=''*, *serial_number=''*, *max_supported_width=None*, *max_supported_height=None*, *max_supported_framerate=None*, *intrinsics=None*, *received_width=0*, *received_height=0*, *received_ratio=0.0*, *received_framerate=0.0*, *max_width=1920*, *max_height=1080*, *max_framerate=30*, *quality=80*, *mimetype='image/jpeg'*, *exif_enabled=True*, *_exif_tags=<factory>*, *_exif_timestamp=0.0*, *_received_frame_at=0.0*, *running=False*, *status='stopped'*, *image=b''*)

Bases: *OverlayMixin*, Position

The network-transparent half of a camera.

Every field here is published over SpiriSynq and can be set by a remote peer. Camera adds the behaviour that reacts to those fields; this class is what a mirror on another node sees.

A camera is a Position too: relative_to is what it is mounted on (a gimbal, an airframe) and mount how, so its pose is where it is and which way it looks, in ECEF. Its own frame is FRD: x out of the lens, y to the picture's right, z down it (see *SpiriCamera.projection*). A camera that relays another's frames (spirisynq://) follows that camera's pose unless it is given its own; one nobody has placed has no pose, and an overlay can't put anything on its picture by where it is in the world.

The other fields fall into five groups:

source_str / source
: What the camera is pointed at. source_str is the request; source is the resolved description, including any error.

vendor / model / serial_number
: Device identity, filled in by the source when it can.

max_supported_*
: What the source reported when it last opened. Writable like everything else here — nothing in SpiriSynq is truly read-only — though a value set by hand does not reconfigure the device and is overwritten the next time the source opens.

max_width / max_height / max_framerate / quality / mimetype / exif_enabled
: The requested capture and encoding settings. These are live: changing them takes effect on the next frame.

received_* / _exif_tags / _exif_timestamp
: What the current image turned out to be. See synq_skip_sync and the note below — none of

this group goes over the wire as a field of its own.

Tags themselves are not a synced field at all. A caller adds them through `Camera.exif_set_tags()` / `Camera.exif_update()` and reads them back from `Camera.exif_tags`, which is recomputed from `image` rather than stored and sent.

Note

`received_*` and the two `_exif_*` fields describe one specific frame, so they must never be published as fields of their own. SpiriSynq syncs each field independently and promises nothing about two of them arriving together or in order, so a width published beside an image is a width a peer can read against the wrong image. They are instead written *into* the encoded frame by *`SpiriCamera.exif`* and read back out of it, on the authoritative node and on every mirror alike, by `Camera._on_image_changed()`. A mirror therefore fills these in for itself and stays exact by construction.

The two `_exif_*` fields carry a leading underscore *and* sit in `synq_skip_sync`: the underscore is what actually keeps them off the wire and out of a full rehydrate (SpiriSynq excludes any field named that way unconditionally), and the explicit skip-set entry is only for `received_*`, which stays public because nothing else needs a leading underscore to keep it from being a field a remote peer could set out from under the image it describes. Read `Camera.exif_tags` instead of touching `_exif_tags` directly; the underscore says "computed for you", not "hidden".

Note

No field here is read-only, and none should be made read-only. Every one of them is reachable over SpiriSynq, so a remote peer can already write any of them; a property setter or a one-way UI binding would only hide that, not prevent it. Some fields are *reported* rather than *requested* — the `max_supported_*` group especially — but "the source overwrites this on open" is a description of behaviour, not a contract that it cannot be written. Being able to set a field to a deliberately wrong value and watch what downstream does with it is a debugging feature worth keeping.

6.1.18 Attributes

source_str
: [str] Requested source URL. Assigning to it retargets the camera.

source
: [SourceInfo] Resolved description of `source_str`.

running
: [bool] Whether the camera is started: capturing, or waiting for a source that will not open yet. Assigning to it starts or stops the camera, including from a remote peer.

status
: [str] What the camera is doing, in words: `STATUS_RUNNING`, `STATUS_STOPPED`, `"<STATUS_WAITING>: <reason>"` for a started camera whose device or upstream topic is not there (yet, or any more), or the reason it is not running — a source string that would not resolve, or a capture that keeps failing. `running` says whether the camera is trying; `status`

says whether frames are actually flowing, and why not when they are not.

image
: [bytes] The most recent encoded frame, in `mimetype` format, carrying that frame's tags in its EXIF; see `Camera.exif_tags_for_frame()`.

exif_enabled
: [bool] Whether to tag encoded frames at all. Turning it off gives byte identical frames for an unchanging scene, which psygnal then suppresses instead of republishing.

intrinsics
: [Intrinsics | None] The lens, from the most trusted of `INTRINSICS_ORIGINS` that has reported one, or `None` when nothing has. Its size is the source's native one where known; scale it to `received_width` x `received_height` to use it on a frame.

`synq_skip_sync = {'received_framerate', 'received_height', 'received_ratio', 'received_width'}`

Fields excluded from SpiriSynq, on top of the base class's own.

Every one of these describes the current `image` – either derived from its bytes directly, or (`received_framerate`) from the local wall-clock gap since the previous one – and is computed fresh wherever it is needed. Publishing it separately would let a peer pair an observation with the wrong frame, or with a framerate measured against a different network path than its own; see the class docstring. `_exif_tags` and `_exif_timestamp` do not need an entry here — their leading underscore already excludes them.

```
source_str: str = ''

source: SourceInfo

vendor: str = ''

model: str = ''

serial_number: str = ''

max_supported_width: int | None = None

max_supported_height: int | None = None

max_supported_framerate: int | None = None

intrinsics: Intrinsics | None = None

received_width: int = 0

received_height: int = 0

received_ratio: float = 0.0

received_framerate: float = 0.0

max_width: int = 1920
```

max_height: *int* = 1080

max_framerate: *int* = 30

quality: *int* = 80

mimetype: *str* = 'image/jpeg'

exif_enabled: *bool* = True

running: *bool* = False

status: *str* = 'stopped'

image: *bytes* = b''

Parameters

- **synq_topic** (*str*)
- **synq_base_topic** (*str | None*)
- **synq_session** (*Session | None*)
- **synq_authoritive** (*bool*)
- **synq_lazy_publish** (*bool*)
- **synq_publish** (*bool*)
- **synq_receive** (*bool*)
- **synq_auto_start** (*bool*)
- **synq_check_receive_types** (*bool*)
- **synq_auto_rehydrate_on_missing_parent_timeout** (*float*)
- **synq_mtime** (*float*)
- **synq_is_deleted** (*bool*)
- **_synq_callbacks** (*dict*)
- **_synq_last_rehydrate_time** (*float*)
- **relative_to** (*RootFrame | Link | str*)
- **mount** (*Pose | None*)
- **pose** (*Pose | None*)
- **_parent_subscriber** (*Subscriber | None*)
- **_parent_pose** (*Pose | None*)
- **_parent_live** (*bool*)
- **_parent_generation** (*int*)

- **overlay_widgets** (*EventedDict*)
- **overlay_client_render** (*bool*)
- **source_str** (*str*)
- **source** (SourceInfo)
- **vendor** (*str*)
- **model** (*str*)
- **serial_number** (*str*)
- **max_supported_width** (*int | None*)
- **max_supported_height** (*int | None*)
- **max_supported_framerate** (*int | None*)
- **intrinsics** (Intrinsics | *None*)
- **received_width** (*int*)
- **received_height** (*int*)
- **received_ratio** (*float*)
- **received_framerate** (*float*)
- **max_width** (*int*)
- **max_height** (*int*)
- **max_framerate** (*int*)
- **quality** (*int*)
- **mimetype** (*str*)
- **exif_enabled** (*bool*)
- **_exif_tags** (*dict[str, str]*)
- **_exif_timestamp** (*float*)
- **_received_frame_at** (*float*)
- **running** (*bool*)
- **status** (*str*)
- **image** (*bytes*)

exception SpiriCamera.**CameraError**

Bases: RuntimeError

Base class for camera-level failures.

exception SpiriCamera.**CameraNotStartedError**

Bases: *CameraError*

Raised when frames are requested from a camera that is not running.

class SpiriCamera.**CaptureSettings**(*max_width=1920*, *max_height=1080*, *max_framerate=30*)

Bases: object

What the camera is asking the source to produce.

These are upper bounds, not exact dimensions: a source is free to deliver something smaller, and reports what it settled on through SourceCapabilities.

> **Note**
>
> Do not shorten these to width / height / framerate. The max_ prefix is the contract: a source may answer a request for 1920x1080 with a 640x480 frame and still be correct. A bare width would read as "the width you will get", which is a promise no source makes. The names also match the *CameraBase* fields they are copied from, so the two stay greppable together.

6.1.19 Attributes

max_width
[int] Largest acceptable frame width in pixels.

max_height
[int] Largest acceptable frame height in pixels.

max_framerate
[int] Highest requested frames per second.

max_width: int = 1920

max_height: int = 1080

max_framerate: int = 30

Parameters

- **max_width** (*int*)
- **max_height** (*int*)
- **max_framerate** (*int*)

exception SpiriCamera.**FrameUnavailableError**

Bases: *CameraError*

Raised when no frame could be produced and none was captured earlier.

class SpiriCamera.**Intrinsics**(*width*, *height*, *fx*, *fy*, *cx*, *cy*, *origin=''*)

Bases: object

A pinhole camera model, at the resolution it was measured at.

The model is in pixels of a `width` x `height` image. A frame of another size with the same aspect – a camera asked for a smaller `max_width`, say – uses `scaled()` to that size, rather than the numbers as they are.

Synced as one value, not field by field, so a peer never sees one report's focal length with another's size.

6.1.20 Attributes

width
[int] Image width the model is in, in pixels.

height
[int] Image height the model is in, in pixels.

fx
[float] Horizontal focal length, in pixels.

fy
[float] Vertical focal length, in pixels.

cx
[float] Principal point, horizontally, in pixels from the left edge.

cy
[float] Principal point, vertically, in pixels from the top edge.

origin
[str] Where it came from: `manual` (the camera's settings), `source` (the source reported it, e.g. a Gazebo sensor's `camera_info`) or `mavlink` (the camera's MAVLink component or autopilot, through SpiriMav). See *INTRINSICS_ORIGINS*.

width: int

height: int

fx: float

fy: float

cx: float

cy: float

origin: str = ''

classmethod from_fov(*width*, *height*, *hfov*, *vfov=0.0*, *origin=''*)

The model of a centred lens with a given field of view.

Parameters

width, height
[int] Image size to express the model in.

hfov
[float] Horizontal field of view, in degrees.

vfov
[float] Vertical field of view, in degrees. `0` assumes square pixels, so it follows from `hfov` and the image's aspect.

origin
[str] See the class attributes.

Parameters

- **width** (*int*)
- **height** (*int*)
- **hfov** (*float*)
- **vfov** (*float*)
- **origin** (*str*)

Return type
Intrinsics

property hfov: float

Horizontal field of view through the principal point, in degrees.

property vfov: float

Vertical field of view through the principal point, in degrees.

scaled(*width*, *height*)

The same lens, for an image resized to `width` x `height`.

Parameters

- **width** (*int*)
- **height** (*int*)

Return type
Intrinsics

Parameters

- **width** (*int*)
- **height** (*int*)
- **fx** (*float*)
- **fy** (*float*)

- **cx** (*float*)
- **cy** (*float*)
- **origin** (*str*)

class SpiriCamera.**SourceBase**(*url*)

Bases: ABC

Abstract base for camera source handlers.

A handler is responsible for its own protocol end to end: deciding whether it recognises a source string, parsing it, opening the underlying device, and producing raw BGR frames. Registration is implicit — *SpiriCamera.sources* discovers every concrete subclass that has been imported.

Subclasses declare the schemes they own via `schemes` and implement `open()`, `close()`, `read()`, and `is_open`. A handler that accepts schemeless strings (such as a V4L2 device path) also overrides `handles()`.

6.1.21 Attributes

schemes
[ClassVar[tuple[str, …]]] URL schemes this handler claims, without the `://` separator.

```
schemes: ClassVar[tuple[str, ...]] = ()
```

classmethod handles(*url*)

Report whether this handler recognises a source URL.

The default implementation matches `schemes`. Override to accept schemeless forms.

Parameters

url
[SourceURL] The parsed source string.

Returns

bool
True if this handler can serve the URL.

Parameters
url (SourceURL)

Return type
bool

classmethod from_url(*url*)

Instantiate this handler for a source URL it has claimed.

Override when a handler needs to validate or normalise the target before construction.

Parameters

url
: [SourceURL] The parsed source string.

Returns

Self
: A handler instance, not yet opened.

Raises

SourceError
: If the target is malformed for this scheme.

Parameters

url (SourceURL)

Return type

Self

property scheme: str

Normalised scheme name, filled in for schemeless sources.

property target: str

device path, stream URL, or image name.

Type

Scheme-specific target

describe()

Summarise this source for the camera's synced state.

Returns

SourceInfo
: A network-transparent description of this handler.

Return type

SourceInfo

abstract property is_open: bool

Whether the underlying device is currently open.

abstractmethod open(*settings*)

Open the underlying device and negotiate capture settings.

Calling `open()` on an already-open source is a no-op that re-reports the current capabilities.

Parameters

settings
: [CaptureSettings] The resolution and framerate the camera is asking for.

Returns

SourceCapabilities
: What the source will actually deliver.

Raises

SourceError
: If the device cannot be opened.

Parameters
: **settings** (CaptureSettings)

Return type
: SourceCapabilities

abstractmethod close()

Release the underlying device.

Closing an already-closed source is a no-op.

Return type
: None

abstractmethod read(*settings*)

Produce a single frame.

Handlers return raw pixels; JPEG encoding and publication belong to the camera layer.

Parameters

settings
: [CaptureSettings] The resolution and framerate currently requested. Sources that render on demand should honour this on every call so that live resolution changes take effect.

Returns

np.ndarray | None
: A BGR frame, or `None` when no new frame is available. Returning the *same* array object on consecutive calls tells the camera the frame is unchanged and lets it reuse its previous encode.

Raises

SourceError
: If the source has failed in a way a retry will not fix.

Parameters
settings (CaptureSettings)

Return type
ndarray | None

frame_tags()

Tags that arrived with the frame `read()` last returned.

For sources whose frames were already tagged before they got here – another camera's output, say – so the metadata survives being decoded and re-encoded. The camera merges these under its own tags; see *exif_tags_for_frame()*.

Returns

dict[str, str]
The inherited tags. The default reports none.

Return type
dict[str, str]

frame_intrinsics()

The lens of the frame `read()` last returned, if it changes.

For sources whose lens can change while open – another camera's output, whose lens zooms – so the camera follows it rather than keeping the one `open()` reported.

Returns

Intrinsics | None
The lens now, or `None` to keep the last one reported. The default keeps what `open()` said.

Return type
Intrinsics | None

Parameters
url (SourceURL)

class SpiriCamera.**SourceCapabilities**(*max_supported_width=None*, *max_supported_height=None*, *max_supported_framerate=None*, *vendor=''*, *model=''*, *serial_number=''*, *intrinsics=None*, *frame=''*)

Bases: `object`

What a source reports it can provide, once opened.

Note

These keep the `max_supported_` prefix for the same reason `CaptureSettings` keeps `max_`: they are ceilings the device advertises, not a guarantee about any particular frame, and they are copied verbatim onto the matching *`CameraBase`* fields.

6.1.22 Attributes

max_supported_width
: [int | None] Largest width the source can deliver, or `None` if it renders at whatever size it is asked for.

max_supported_height
: [int | None] Largest height the source can deliver, or `None` if it is resolution independent.

max_supported_framerate
: [int | None] Highest framerate the source can deliver, or `None` if unconstrained.

vendor
: [str] Device vendor, when the source can determine one.

model
: [str] Device model, when the source can determine one.

serial_number
: [str] Device serial number, when the source can determine one.

intrinsics
: [Intrinsics | None] The lens, when the source knows it – a simulated camera does, exactly; most devices don't say. Its `origin` is filled in by the camera.

frame
: [str] The SpiriSynq topic of the `Position` the frames are seen from, when they are some other object's view: a relayed camera's. The camera follows its pose unless it has been placed itself. Empty for a source that is the camera.

```
max_supported_width: int | None = None

max_supported_height: int | None = None

max_supported_framerate: int | None = None

vendor: str = ''

model: str = ''

serial_number: str = ''

intrinsics: Intrinsics | None = None

frame: str = ''
```

Parameters

- **max_supported_width** (*int | None*)

- **max_supported_height** (*int | None*)
- **max_supported_framerate** (*int | None*)
- **vendor** (*str*)
- **model** (*str*)
- **serial_number** (*str*)
- **intrinsics** (Intrinsics | *None*)
- **frame** (*str*)

exception SpiriCamera.**SourceError**

Bases: Exception

Base class for source resolution, open, and read failures.

class SpiriCamera.**SourceInfo**(*url=''*, *scheme=''*, *target=''*, *handler=''*, *error=''*)

Bases: SubSyncableDataclass

Network-transparent description of a camera's resolved source.

This is the source-shaped half of the camera's synced state: it travels over SpiriSynq so a remote peer can see what a camera is pointed at without having to parse the source string itself.

6.1.23 Attributes

url
[str] The source string this description was resolved from.

scheme
[str] Normalised scheme (`v4l`, `rtsp`, `testimage`, …).

target
[str] Scheme-specific target: device path, stream URL, or image name.

handler
[str] Name of the handler class that claimed the source.

error
[str] Empty when the source resolved cleanly, otherwise the reason it could not be resolved.

url: **str** = ''

scheme: **str** = ''

target: **str** = ''

handler: **str** = ''

error: **str** = ''

classmethod unresolved(*url*, *error*)

Build a description for a source string that could not be resolved.

Parameters

url
: [str] The offending source string.

error
: [str] Human-readable reason the source was rejected.

Returns

SourceInfo
: A description carrying only `url` and `error`.

Parameters

- **url** (*str*)
- **error** (*str*)

Return type
: SourceInfo

update(*other*)

Copy another description's fields into this one, field by field.

Updating in place keeps the object identity stable, so UI bindings and sync subscriptions attached to this instance survive a source change, and only the fields that actually changed are published.

Parameters

other
: [SourceInfo] The description to copy from.

Parameters
: **other** (SourceInfo)

Return type
: None

Parameters

- **url** (*str*)
- **scheme** (*str*)
- **target** (*str*)
- **handler** (*str*)
- **error** (*str*)

class SpiriCamera.**SourceURL**(*raw*, *scheme*, *target*)

Bases: object

A source string split into its scheme and target.

6.1.24 Attributes

raw
: [str] The original (stripped) source string.

scheme
: [str] Lowercased scheme, or `""` for schemeless strings such as `/dev/video0` or a bare camera index.

target
: [str] Everything after `://`, or the whole string when schemeless.

raw: *str*

scheme: *str*

target: *str*

classmethod parse(*source_str*)

Split a source string into scheme and target.

Parameters

source_str
: [str] The source string to parse.

Returns

SourceURL
: The split representation.

Raises

SourceError
: If the source string is empty or whitespace only.

Parameters
: **source_str** (*str*)

Return type
: SourceURL

Parameters

- **raw** (*str*)
- **scheme** (*str*)
- **target** (*str*)

exception SpiriCamera.**UnknownSourceError**

Bases: *SourceError*

Raised when no registered handler recognises a source string.

SpiriCamera.**describe_source**(*source_str*)

Resolve a source string and describe it without opening anything.

6.1.25 Parameters

source_str
[str] The source string to inspect.

6.1.26 Returns

SourceInfo
A network-transparent description of the resolved source.

6.1.27 Raises

SourceError
If the source string cannot be resolved.

Parameters
source_str (*str*)

Return type
SourceInfo

SpiriCamera.**known_schemes**()

List the URL schemes the registered handlers claim.

6.1.28 Returns

list[str]
Sorted scheme names, formatted as `scheme://`.

Return type
list[str]

SpiriCamera.**registered_sources**()

List every concrete source handler currently importable.

6.1.29 Returns

list[type[SourceBase]]
Concrete `SourceBase` subclasses, depth-first.

Return type
list[type[SourceBase]]

SpiriCamera.**resolve_source**(*source_str*)

Find and instantiate the handler for a source string.

Handlers that claim the URL's explicit scheme win over handlers that match schemeless strings, so `v4l://0` and `/dev/video0` both land on the V4L2 handler without either rule shadowing the other.

6.1.30 Parameters

source_str
[str] Source string such as `/dev/video0`, `rtsp://host/stream`, or `testimage://pm5544`.

6.1.31 Returns

SourceBase
A handler instance, not yet opened.

6.1.32 Raises

SourceError
If the source string is empty or malformed for its scheme.

UnknownSourceError
If no registered handler recognises the source string.

Parameters
source_str (*str*)

Return type
SourceBase

6.2 Modules

6.2.1 SpiriCamera.camera

The camera domain: capture lifecycle, encoding, and publication.

A *Camera* owns *when* frames are taken, *how* they are encoded, and *where* they are published. It owns none of the protocol detail of getting hold of a frame — that belongs to the handlers in *SpiriCamera.sources*, which hand back raw BGR arrays and never reach back into the camera.

SpiriCamera.camera.**ENCODINGS: dict[str, str] = {'image/jpeg': '.jpg'}**
Mimetypes the camera can encode to, mapped to an OpenCV extension.

JPEG only. It is the format every consumer of this package already decodes, it is the only one the frame tags in *SpiriCamera.exif* are written for, and offering a second one bought nothing but a second path to test.

SpiriCamera.camera.**STATUS_RUNNING = 'running'**
CameraBase.status while frames are being captured.

SpiriCamera.camera.**STATUS_STOPPED = 'stopped'**
CameraBase.status while idle with nothing wrong.

SpiriCamera.camera.**STATUS_WAITING = 'waiting for source'**
Prefix of *CameraBase.status* while running but waiting for a source that will not open yet; the reason follows after a colon.

SpiriCamera.camera.**INTRINSICS_ORIGINS = ('manual', 'source', 'mavlink')**

Where *CameraBase.intrinsics* can come from, most trusted first: what someone configured beats what the source reports, which beats what a MAVLink camera or autopilot says (someone typed those numbers in somewhere too, just further from the camera).

exception SpiriCamera.camera.**CameraError**

Bases: RuntimeError

Base class for camera-level failures.

exception SpiriCamera.camera.**CameraNotStartedError**

Bases: *CameraError*

Raised when frames are requested from a camera that is not running.

exception SpiriCamera.camera.**FrameUnavailableError**

Bases: *CameraError*

Raised when no frame could be produced and none was captured earlier.

SpiriCamera.camera.**topic_for_source**(*source_str*)

Derive a stable SpiriSynq topic from a source string.

Parameters

source_str
[str] The camera's initial source string.

Returns

str
A topic-safe name such as `spiricamera_dev_video0`.

Parameters
source_str (*str*)

Return type
str

class SpiriCamera.camera.**CameraBase**(*synq_topic*, *synq_base_topic=None*, *synq_session=<factory>*, *synq_authoritive=False*, *synq_lazy_publish=False*, *synq_publish=True*, *synq_receive=True*, *synq_auto_start=True*, *synq_check_receive_types=True*, *synq_auto_rehydrate_on_missing_parent_timeout=5.0*, *synq_mtime=-1*, *synq_is_deleted=False*, *_synq_callbacks=<factory>*, *_synq_last_rehydrate_time=0.0*, *relative_to='unknown'*, *mount=None*, *pose=None*, *_parent_subscriber=None*, *_parent_pose=None*, *_parent_live=False*, *_parent_generation=0*, *overlay_widgets=<factory>*, *overlay_client_render=False*, *source_str=''*, *source=<factory>*, *vendor=''*, *model=''*, *serial_number=''*, *max_supported_width=None*, *max_supported_height=None*, *max_supported_framerate=None*, *intrinsics=None*, *received_width=0*, *received_height=0*, *received_ratio=0.0*, *received_framerate=0.0*, *max_width=1920*, *max_height=1080*, *max_framerate=30*, *quality=80*, *mimetype='image/jpeg'*, *exif_enabled=True*, *_exif_tags=<factory>*, *_exif_timestamp=0.0*, *_received_frame_at=0.0*, *running=False*, *status='stopped'*, *image=b''*)

Bases: *OverlayMixin*, Position

The network-transparent half of a camera.

Every field here is published over SpiriSynq and can be set by a remote peer. *Camera* adds the behaviour that reacts to those fields; this class is what a mirror on another node sees.

A camera is a Position too: relative_to is what it is mounted on (a gimbal, an airframe) and mount how, so its pose is where it is and which way it looks, in ECEF. Its own frame is FRD: x out of the lens, y to the picture's right, z down it (see *SpiriCamera.projection*). A camera that relays another's frames (spirisynq://) follows that camera's pose unless it is given its own; one nobody has placed has no pose, and an overlay can't put anything on its picture by where it is in the world.

The other fields fall into five groups:

source_str / source
: What the camera is pointed at. source_str is the request; source is the resolved description, including any error.

vendor / model / serial_number
: Device identity, filled in by the source when it can.

max_supported_*
: What the source reported when it last opened. Writable like everything else here — nothing in SpiriSynq is truly read-only — though a value set by hand does not reconfigure the device and is overwritten the next time the source opens.

max_width / max_height / max_framerate / quality / mimetype / exif_enabled

The requested capture and encoding settings. These are live: changing them takes effect on the next frame.

`received_*` / `_exif_tags` / `_exif_timestamp`
What the current *`image`* turned out to be. See *`synq_skip_sync`* and the note below — none of this group goes over the wire as a field of its own.

Tags themselves are not a synced field at all. A caller adds them through *`Camera.exif_set_tags()`* / *`Camera.exif_update()`* and reads them back from *`Camera.exif_tags`*, which is recomputed from *`image`* rather than stored and sent.

Note

`received_*` and the two `_exif_*` fields describe one specific frame, so they must never be published as fields of their own. SpiriSynq syncs each field independently and promises nothing about two of them arriving together or in order, so a width published beside an image is a width a peer can read against the wrong image. They are instead written *into* the encoded frame by *`SpiriCamera.exif`* and read back out of it, on the authoritative node and on every mirror alike, by `Camera._on_image_changed()`. A mirror therefore fills these in for itself and stays exact by construction.

The two `_exif_*` fields carry a leading underscore *and* sit in `synq_skip_sync`: the underscore is what actually keeps them off the wire and out of a full rehydrate (SpiriSynq excludes any field named that way unconditionally), and the explicit skip-set entry is only for `received_*`, which stays public because nothing else needs a leading underscore to keep it from being a field a remote peer could set out from under the image it describes. Read *`Camera.exif_tags`* instead of touching `_exif_tags` directly; the underscore says "computed for you", not "hidden".

Note

No field here is read-only, and none should be made read-only. Every one of them is reachable over SpiriSynq, so a remote peer can already write any of them; a property setter or a one-way UI binding would only hide that, not prevent it. Some fields are *reported* rather than *requested* — the `max_supported_*` group especially — but "the source overwrites this on open" is a description of behaviour, not a contract that it cannot be written. Being able to set a field to a deliberately wrong value and watch what downstream does with it is a debugging feature worth keeping.

Attributes

source_str
[str] Requested source URL. Assigning to it retargets the camera.

source
[SourceInfo] Resolved description of *`source_str`*.

running
[bool] Whether the camera is started: capturing, or waiting for a source that will not open yet. Assigning to it starts or stops the camera, including from a remote peer.

status

[str] What the camera is doing, in words: *STATUS_RUNNING*, *STATUS_STOPPED*, "<STATUS_WAITING>: <reason>" for a started camera whose device or upstream topic is not there (yet, or any more), or the reason it is not running — a source string that would not resolve, or a capture that keeps failing. `running` says whether the camera is trying; `status` says whether frames are actually flowing, and why not when they are not.

image

[bytes] The most recent encoded frame, in *mimetype* format, carrying that frame's tags in its EXIF; see *Camera.exif_tags_for_frame()*.

exif_enabled

[bool] Whether to tag encoded frames at all. Turning it off gives byte identical frames for an unchanging scene, which psygnal then suppresses instead of republishing.

intrinsics

[Intrinsics | None] The lens, from the most trusted of *INTRINSICS_ORIGINS* that has reported one, or `None` when nothing has. Its size is the source's native one where known; scale it to `received_width` x `received_height` to use it on a frame.

synq_skip_sync = {'received_framerate', 'received_height', 'received_ratio', 'received_width'}

Fields excluded from SpiriSynq, on top of the base class's own.

Every one of these describes the current *image* – either derived from its bytes directly, or (`received_framerate`) from the local wall-clock gap since the previous one – and is computed fresh wherever it is needed. Publishing it separately would let a peer pair an observation with the wrong frame, or with a framerate measured against a different network path than its own; see the class docstring. `_exif_tags` and `_exif_timestamp` do not need an entry here — their leading underscore already excludes them.

```
source_str: str = ''

source: SourceInfo

vendor: str = ''

model: str = ''

serial_number: str = ''

max_supported_width: int | None = None

max_supported_height: int | None = None

max_supported_framerate: int | None = None

intrinsics: Intrinsics | None = None

received_width: int = 0

received_height: int = 0
```

```
received_ratio: float = 0.0

received_framerate: float = 0.0

max_width: int = 1920

max_height: int = 1080

max_framerate: int = 30

quality: int = 80

mimetype: str = 'image/jpeg'

exif_enabled: bool = True

running: bool = False

status: str = 'stopped'

image: bytes = b''
```

Parameters

- **synq_topic** (*str*)
- **synq_base_topic** (*str | None*)
- **synq_session** (*Session | None*)
- **synq_authoritive** (*bool*)
- **synq_lazy_publish** (*bool*)
- **synq_publish** (*bool*)
- **synq_receive** (*bool*)
- **synq_auto_start** (*bool*)
- **synq_check_receive_types** (*bool*)
- **synq_auto_rehydrate_on_missing_parent_timeout** (*float*)
- **synq_mtime** (*float*)
- **synq_is_deleted** (*bool*)
- **_synq_callbacks** (*dict*)
- **_synq_last_rehydrate_time** (*float*)
- **relative_to** (*RootFrame | Link | str*)
- **mount** (*Pose | None*)
- **pose** (*Pose | None*)
- **_parent_subscriber** (*Subscriber | None*)

- **_parent_pose** (*Pose | None*)
- **_parent_live** (*bool*)
- **_parent_generation** (*int*)
- **overlay_widgets** (*EventedDict*)
- **overlay_client_render** (*bool*)
- **source_str** (*str*)
- **source** (SourceInfo)
- **vendor** (*str*)
- **model** (*str*)
- **serial_number** (*str*)
- **max_supported_width** (*int | None*)
- **max_supported_height** (*int | None*)
- **max_supported_framerate** (*int | None*)
- **intrinsics** (Intrinsics | *None*)
- **received_width** (*int*)
- **received_height** (*int*)
- **received_ratio** (*float*)
- **received_framerate** (*float*)
- **max_width** (*int*)
- **max_height** (*int*)
- **max_framerate** (*int*)
- **quality** (*int*)
- **mimetype** (*str*)
- **exif_enabled** (*bool*)
- **_exif_tags** (*dict[str, str]*)
- **_exif_timestamp** (*float*)
- **_received_frame_at** (*float*)
- **running** (*bool*)
- **status** (*str*)
- **image** (*bytes*)

class SpiriCamera.camera.**Camera**(*source=''*, ***, *quality=80*, *max_width=1920*, *max_height=1080*, *max_framerate=30*, *mimetype='image/jpeg'*, *synq_topic=''*, *synq_auto_start=True*, *synq_authoritive=None*, ***rehydrated*)

Bases: *CameraBase*

A live camera: resolves a source, captures frames, publishes them.

Usage:

```
cam = Camera("/dev/video0", quality=85)
cam.start()                  # begins the background capture thread
frame_bytes = cam.image      # whatever it has captured so far
cam.stop()
```

or as a context manager, which starts on entry and stops on exit:

```
with Camera("testimage://", max_width=1280, max_height=720) as cam:
    frame_bytes = cam.image
```

Treat a running `Camera` like a small daemon, not a handle to poll by calling *`read()`* yourself: leave `start()` on its default `background=True` and let the capture thread run its own loop, paced to *`max_framerate`*. To change the rate, set `max_framerate` — it is live and takes effect on the next frame — rather than calling *`read()`* less often. `start(background=False)` exists for scripted, frame-at-a-time use (tests, one-off captures); it is not the pattern for anything long-running.

This is the *authoritative* role: actually owning and driving a device. Code that merely wants another process's frames should not construct a second `Camera` this way — it should mirror the existing one with `from_topic()` and either poll *`image`* or connect to its psygnal event (`cam.events.image.connect(...)`); see the "Consuming an existing camera" section of the getting-started guide.

While running, a background thread captures at *`max_framerate`* and assigns each encoded frame to *`image`*, which publishes it to SpiriSynq. Settings are live: changing `max_width`, `quality`, or `max_framerate` — locally or from a remote peer — takes effect on the next frame without a restart.

An unusable source is not fatal. Construction never raises for a bad source string; the reason lands in `camera.source.error` and *`start()`* is what refuses. This keeps the camera bindable to a live-edited input field.

Parameters

- **source** (SourceInfo)
- **quality** (*int*)
- **max_width** (*int*)
- **max_height** (*int*)
- **max_framerate** (*int*)
- **mimetype** (*str*)
- **synq_topic** (*str*)
- **synq_auto_start** (*bool*)

- **synq_authoritive** (*bool*)
- **rehydrated** (*object*)

property handler: *SourceBase* | None

The resolved source handler, or `None` if unresolvable.

start(*, *background=True*)

Open the source and begin capturing.

Starting an already-running camera is a no-op.

Parameters

background

[bool] Whether to run a capture thread. With `False` the camera opens but captures nothing until the caller drives *`read()`* — useful for scripted, frame-at-a-time use.

A source that resolves but will not open yet – a device that is not plugged in, a `spirisynq://` topic nobody publishes yet – does not stop a background camera from starting. It starts *waiting*: `running` is true, *`status`* reads `"waiting for source: <reason>"`, and the capture thread keeps trying to open the source until it works. The same happens if the source is lost while running.

Raises

CameraError

If the source string could not be resolved, or – only with `background=False`, where no thread is left to wait – the source refused to open.

Parameters

background (*bool*)

Return type

None

stop()

Stop capturing and release the source.

Stopping an already-stopped camera is a no-op. Safe to call from the capture thread itself, in which case the thread is not joined.

Return type

None

close()

Stop the camera and release its SpiriSynq resources.

Return type

None

webrtc_offer(*sdp*, *type='offer'*)

Negotiate a WebRTC session publishing this camera's frames.

The zenoh-native counterpart to *SpiriCamera.whep*'s HTTP route: both end up in *SpiriCamera.webrtc.negotiate_sync()*, so a viewer reached either way gets the same video track and tags channel.

Parameters

sdp
[str] The remote peer's SDP offer.

type
[str] The offer's SDP type, almost always `"offer"`.

Returns

dict[str, str]
`{"session_id": ..., "sdp": ..., "type": "answer"}`.

Parameters

- **sdp** (*str*)
- **type** (*str*)

Return type
dict[str, str]

set_intrinsics(*origin*, *width=0*, *height=0*, *fx=0.0*, *fy=0.0*, *cx=0.0*, *cy=0.0*, *hfov=0.0*, *vfov=0.0*)

Report this camera's lens, as *origin* knows it.

Either a full pinhole model – `width`, `height`, `fx` and `fy`, with `cx`/`cy` defaulting to the image centre – or just a field of view in degrees, `hfov` and optionally `vfov`, which becomes a centred model at the camera's own frame size once it knows one. A later report from the same origin replaces this one; *intrinsics* is whichever origin ranks highest in *INTRINSICS_ORIGINS*.

Plain numbers rather than an `Intrinsics`, so a peer that doesn't have this package (SpiriMav) can call it.

Raises

ValueError
For an unknown origin, or numbers that aren't a lens.

Parameters

- **origin** (*str*)
- **width** (*int*)
- **height** (*int*)
- **fx** (*float*)

- **fy** (*float*)
- **cx** (*float*)
- **cy** (*float*)
- **hfov** (*float*)
- **vfov** (*float*)

Return type
None

clear_intrinsics(*origin*)

Forget what *origin* reported, falling back to the next one.

Parameters
origin (*str*)

Return type
None

capture_settings()

Snapshot the currently requested capture settings.

Values arriving from UI bindings or remote peers may be floats or `None`; they are coerced to positive integers here so sources can trust what they are given.

Returns

CaptureSettings
The settings to hand to the source.

Return type
CaptureSettings

describe_capabilities()

Summarise what the source reported, for logs and the CLI.

Returns

str
Something like `1920x1080 @ 30fps`. Sources that render at whatever size they are asked for report no fixed limits, and are described as `any size` / `any rate`.

Return type
str

read_frame()

Grab one raw frame from the source.

Returns

np.ndarray | None
A BGR frame, or `None` if the source had nothing new.

Raises

CameraNotStartedError
If the camera is not running.

SourceError
If the source failed unrecoverably.

Return type
ndarray | None

read()
Capture, encode, and publish a single frame.

The encoded frame is tagged, then assigned to *image*, which is what publishes it to SpiriSynq. Assigning is also what fills in *received_width* and the rest, since those are read back out of the frame rather than measured here.

Returns

bytes
The encoded frame, in *mimetype* format. If the source had no new frame, the previous one is returned unchanged.

Raises

CameraNotStartedError
If the camera is not running.

FrameUnavailableError
If the source produced nothing and no earlier frame exists.

Return type
bytes

property exif_tags: dict[str, str]
The tags on the current *image*.

Read back out of the frame itself rather than stored, so this is never out of step with what is actually on screen — on the authoritative node or on any mirror, since both learn it the same way, from `_on_image_changed()`.

property exif_timestamp: float
Unix timestamp the current *image* was captured at, or `0.0` if it carried none.

Set by whichever node took the frame, so comparing it against a local clock is only as good as the agreement between the two.

exif_set_tags(*provider*, *tags*)

Replace one provider's tags, leaving every other provider's.

This is the primitive for more than one piece of software tagging the same camera's frames without stepping on each other: each provider owns its own bucket, keyed by `provider`, and setting one never touches another's. Call it again with the same `provider` to replace what it set before.

Parameters

provider

[str] A name identifying who is setting these tags — your module or component name is a reasonable choice. Two providers using two different names can never clobber one another, no matter what tags they choose or when they call this.

tags

[Mapping[str, str]] The complete set of tags this provider wants embedded in every frame from now on. An empty mapping is the same as *exif_clear_tags()*. Unlike *exif_update()*, a value of `""` is kept rather than dropped, which lets a provider that genuinely needs to blank out a built-in tag — omitting `source` from a frame headed somewhere public, say — do exactly that.

Parameters

- **provider** (*str*)
- **tags** (*Mapping[str, str]*)

Return type

None

exif_clear_tags(*provider*)

Remove everything `provider` has set, and nothing else.

Parameters

provider

[str] The same name passed to *exif_set_tags()*. Clearing a provider that never set anything is not an error.

Parameters

provider (*str*)

Return type

None

exif_tag_providers()

Names of the providers currently contributing custom tags.

Returns

frozenset[str]
Provider names with at least one tag set right now. Does not include the built-in tags `exif_tags_for_frame()` adds on its own, which have no provider name.

Return type
frozenset[str]

exif_update(***tags*)

Merge tags into the default, unscoped tag bucket.

A convenience for the common case of one caller adding a tag or two without needing a provider name of its own. Two *different* callers doing this concurrently still share this one bucket by construction and can still overwrite each other; once that matters, give each caller its own name and use `exif_set_tags()` instead.

Parameters

****tags**
[str] Tags to add or replace in the default bucket. A value of `""` drops that one tag from the default bucket, leaving the rest of it, and every other provider, untouched.

Parameters
tags (*str*)

Return type
None

exif_tags_for_frame(*frame*)

Build the tags to write into the next encoded frame.

The override point for tagging. A subclass with a GPS fix or a gimbal angle to attach on every frame should extend what this returns; a caller or another piece of software adding metadata at runtime should use `exif_set_tags()` / `exif_update()` instead, which this merges in afterwards.

Tags come in three layers, each overriding the one before: tags the source says arrived with the frame (`frame_tags()`, e.g. an upstream camera's GPS tags through a `spirisynq://` source), then this camera's built-ins, then providers. The built-ins describe the node that produced *these* bytes, so they replace an upstream's `source` and `topic` – except the capture time, which is kept from the inherited tags when they carry one.

Providers are merged in over the built-ins in sorted-name order, so the result does not depend on which provider happened to register first — but two providers naming the *same* tag will still resolve in that fixed order regardless, which is worth avoiding by prefixing tag names with something distinguishing rather than relying on.

Dimensions are deliberately absent: the JPEG header already states them, and `_on_image_changed()` reads them from there. A tag saying something the container also says is a tag that can disagree with it.

Parameters

frame
[np.ndarray] The raw BGR frame about to be encoded.

Returns

dict[str, str]
Tags to embed. Empty means embed nothing.

Parameters
frame (*ndarray*)

Return type
dict[str, str]

6.2.2 SpiriCamera.exif

Frame metadata carried *inside* the encoded frame.

Anything that describes a particular frame has to travel with that frame. SpiriSynq publishes each field of a `SyncableObject` independently and makes no promise that two of them arrive together or in order, so a tag published beside an image is a tag that can be read against the wrong image. Baking the tags into the image bytes removes the question: there is one payload, and it either arrived or it did not.

The consequence for the camera domain is that tags are *written* on the authoritative side and *read* on every side, including the authoritative one. Nothing measured off a frame is ever sent as its own field; see the `synq_skip_sync` set on *`CameraBase`*.

Tags are free-form `str` to `str`. The whole mapping is serialised as JSON into the EXIF `UserComment` field, which is what *`extract()`* reads back, so an arbitrary key round-trips exactly. Keys in *`STANDARD_TAGS`* are *additionally* written to the real EXIF tag of the same meaning, so that ordinary image tools see a sensible Make and Model; *`extract()`* falls back to reading those when an image has no `UserComment`, which is what lets tags be recovered from a JPEG this package did not write.

`SpiriCamera.exif.`**`DATETIME_TAG = 'datetime'`**

Tag holding the same instant as EXIF's `YYYY:MM:DD HH:MM:SS` text.

Redundant with *`TIMESTAMP_TAG`* and derived from it, so that ordinary image tools see a date in the field they look in. Whole seconds and no timezone, which is all the EXIF tag holds; read *`TIMESTAMP_TAG`* for the real value.

`SpiriCamera.exif.`**`STANDARD_TAGS: dict[str, tuple[str, int]] = {'datetime': ('0th', 306), 'description': ('0th', 270), 'make': ('0th', 271), 'model': ('0th', 272), 'serial_number': ('Exif', 42033), 'software': ('0th', 305)}`**

Tags that also map onto a real EXIF tag, as `(ifd, tag id)`.

Written in both places; read back from here only when an image carries no JSON `UserComment` to read instead.

`SpiriCamera.exif.`**`TIMESTAMP_TAG = 'timestamp'`**

Tag holding the capture time as a Unix timestamp, in seconds.

A wall clock rather than `time.monotonic()`, because a tagged frame is meant to be read on a different node than the one that took it and a monotonic reading means nothing once it leaves the process that made it. The cost is the usual one: it can step, and two nodes only agree to the accuracy of whatever is keeping their clocks together.

exception SpiriCamera.exif.**ExifError**

Bases: `ValueError`

Raised when image data cannot be tagged.

SpiriCamera.exif.**embed**(*data*, *tags*)

Return `data` with `tags` written into its EXIF.

Any EXIF already present is replaced, so re-tagging a frame does not accumulate segments. Data that is not a JPEG is returned unchanged rather than raising: tagging is a best effort layered on top of whatever the camera was asked to encode.

Parameters

data
[bytes] Encoded image data.

tags
[Mapping[str, str]] Tags to write. An empty mapping returns `data` unchanged.

Returns

bytes
The tagged image data.

Raises

ExifError
If the data is a JPEG but could not be tagged.

Parameters

- **data** (*bytes*)
- **tags** (*Mapping[str, str]*)

Return type

bytes

SpiriCamera.exif.**extract**(*data*)

Read the tags out of image data.

Never raises: data with no tags, damaged EXIF, or a format that carries none all read as no tags. A caller has an image either way, and refusing to show it because its metadata is malformed helps nobody.

Parameters

data
: [bytes] Encoded image data.

Returns

dict[str, str]
: The tags found, empty if there were none.

Parameters

data (*bytes*)

Return type

dict[str, str]

SpiriCamera.exif.**image_size**(*data*)

Read pixel dimensions from a JPEG header, without decoding it.

Not EXIF: this is the container's own declaration of its size, which is the authoritative answer for what was actually encoded and costs a marker walk rather than a decode.

Parameters

data
: [bytes] Encoded image data.

Returns

tuple[int, int] | None
: Width and height, or `None` if the header could not be read.

Parameters

data (*bytes*)

Return type

tuple[int, int] | None

SpiriCamera.exif.**is_jpeg**(*data*)

Whether image data is a JPEG, and so can carry EXIF.

Parameters

data
: [bytes] Encoded image data.

Returns

bool
True if the data starts with the JPEG magic number.

Parameters
data (*bytes*)

Return type
bool

6.2.3 SpiriCamera.overlay

SVG HUD overlays, baked into frames before they are encoded.

An overlay is a *HudWidget*: a small SVG template, anchored to a corner or a percentage coordinate of the frame. Widgets are not owned by the camera that draws them – each is its own `SyncableObject`, published wherever its author runs (a UI panel, an ML detector, …), and *OverlayMixin* only mirrors the ones a camera's `overlay_widgets` names.

A widget's SVG is templated with MiniJinja against three namespaces: `exif` (the current frame's tags, see *SpiriCamera.exif*), `frame` (the current frame's actual, as-received width/height/framerate – see *FRAME_NAME*), and `objects` (the latest field values of whatever other SpiriSynq objects the widget declares for itself). `frame` exists alongside `objects` rather than folded into it because these values – what the *rendered* frame actually is, this call – are never published as synced fields at all (`CameraBase.synq_skip_sync`, see *camera_metrics_widget()*); there is no topic a widget could declare to reach them, only the local values `OverlayMixin._render_overlays()` already has in hand for the frame it is about to composite onto.

A real topic path is never written directly into a `{{ }}` expression – only an alias is, as `{{ objects.<alias>.<field> }}`, because a zenoh topic can contain characters (a hyphen in a hostname, say) that are not valid there. The alias, and the *default* object it resolves to, are declared right next to where the alias is used, as a MiniJinja comment MiniJinja itself already strips at render time and that never appears in the rendered SVG:

```
{# object: battery = azrael/battery_monitor #}
<text>{{ objects.battery.voltage }}</text>
```

A declared object is a whole `SyncableObject` – every one of its synced fields is available as `objects.<alias>.<field>`, mirrored generically via `from_topic_untyped()` (so the widget author needs no locally-importable class for it, and never sees its RPC methods – `from_topic_untyped` only ever exposes synced attributes). This is templated in *resolve_objects()*, which reads both halves of that same text – MiniJinja's own static analysis (`undeclared_variables_in_str`) for what the render body actually reads, against *declared_objects()* for what has been declared – and raises if a template reads an alias with no matching declaration, which is meant to be caught by whoever is authoring the widget, not silently ignored.

The declared topic is only a *default*. Whichever camera is actually rendering the widget can override it – to point `battery` at a different concrete object than the widget author had in mind – via *OverlayMixin.overlay_widgets*, without touching the shared *HudWidget* itself (which would repoint it for every other camera mirroring the same widget too). The same field also carries a per-camera position override, for exactly the same reason: a widget's own `anchor`/`custom_anchor_x`/`custom_anchor_y` are its default placement, and a camera can place it elsewhere for itself alone. See *OverlayMixin.overlay_widgets* for the shape of both.

A widget can also be placed by where its objects are in the world rather than on the frame: `{# anchor: <alias> #}` centres it on an object's pixel, `{# anchor: each <alias>.<field> #}` draws a copy on each object that field links to, and `{# project: <alias>, ... #}` gives a frame-wide widget its objects' pixels. Those widgets get a `view` namespace (and `item`, for `each`), computed by *SpiriCamera.projection* from the camera's own pose and lens; see *world_placement()* and *Drawing on things in the world*.

Rendered SVG is rasterized with `thorvg_python` at its own **natural** pixel size (`Picture.get_size()`), never scaled to fit a declared box: a widget author who writes `font-size="24"` gets a real 24px on screen regardless of the camera's resolution. Scaling a small raster up to fit a 4K frame (or down to fit a low-res one) would either blur the text or make it illegible – crisp, fixed-size text was chosen over staying a constant fraction of the frame across resolutions.

The one deliberate exception is a root `<svg>` whose `width` and `height` are *both* percentages (`width="20%" height="20%"`): that scales the whole widget, preserving its own aspect ratio, to fit within that percentage of the frame being rendered onto – see *rasterize()*. This is opt-in, not a default: a widget author who wants "always a fifth of the picture, whatever that picture's resolution" now has a way to say so, alongside the natural-pixel-size behaviour above, which stays the default for a plain `width="260"`. Which is right depends on the widget – fixed-size text and a HUD panel generally want to stay crisp and constant-sized; a logo, icon, or other graphic that is meant to track the frame's proportions wants percentage sizing instead.

`SpiriCamera.overlay.`**`ANCHORS: tuple[str, ...] = ('top_left', 'top_middle', 'top_right', 'center_left', 'center_middle', 'center_right', 'bottom_left', 'bottom_middle', 'bottom_right', 'custom')`**

> Every valid *HudWidget.anchor* value, named anchors plus `"custom"` – for a UI to offer as a choice, since `_NAMED_ANCHORS` alone would leave "custom" out.

`SpiriCamera.overlay.`**`DEFAULT_FONT = 'Miracode'`**

> The name every bundled-font `<text font-family="...">` in this module's own templates uses, and the only font guaranteed to be available regardless of the host – see the loading block below. Not an arbitrary label: `font_load()` registers a font under the family name baked into the file's own metadata, so this has to match Miracode's actual name table entry, verified empirically below.

`SpiriCamera.overlay.`**`DEFAULT_FONT_PATH = PosixPath('/home/runner/work/SpiriCamera/SpiriCamera/src/SpiriCamera/assets/Miracode.ttf')`**

> Bundled rather than discovered from the host's installed fonts: thorvg does not fall back to any OS text rendering – an SVG *<text>* element with no matching loaded font simply draws nothing, on every platform, including the slim Docker image this project ships (which has no fonts installed at all). A widget author who forgets to reference a loaded font by name gets a silently blank widget, which is exactly the failure this avoids for SpiriCamera's own templates.
>
> Loaded from a real file path (*font_load*), not *font_load_data* from in-memory bytes – this build of thorvg returns *Result.NOT_SUPPORTED* for *font_load_data* regardless of the mimetype string given it, but the file-path loader works. Simpler to resolve the packaged asset's real on-disk path directly than to route around that through *importlib.resources.as_file()*'s context-manager lifetime for what is a one-time, process-lifetime registration anyway.
>
> Miracode was picked for legibility at the small sizes an overlay renders at, not for completeness – it is missing the Ukrainian letters U+0490/U+0491 (Ge with upturn). A widget that needs those specifically should load and reference its own font instead of relying on this default; see

thorvg_python.paint.text.Text.font_load.

font_load() is a global registration despite living on a *Text* instance – the instance itself is discarded immediately, only used to reach the underlying (process-wide, thorvg-internal) font cache. Public so a client-render path (*OverlayMixin.render_overlays_for_client()*) can embed the same file as a browser `@font-face`, keeping the browser's rendering of `DEFAULT_FONT` visually identical to thorvg's rather than falling back to whatever generic font the browser picks for an unresolvable `font-family`.

`SpiriCamera.overlay.`**`EXIF_NAME = 'exif'`**

Top-level template name the current frame's EXIF tags are exposed under.

`SpiriCamera.overlay.`**`FRAME_NAME = 'frame'`**

Top-level template name the current frame's actual, as-received `width`/`height`/`framerate` are exposed under, e.g. `{{ frame.width }}x{{ frame.height }}`. See the module docstring for why these are injected directly rather than reached through `objects`.

`SpiriCamera.overlay.`**`ITEM_NAME = 'item'`**

Top-level template name for the one linked object an `{# anchor: each <alias>.<field> #}` widget is being drawn for, its fields as `{{ item.<field> }}`.

`SpiriCamera.overlay.`**`OBJECTS_NAME = 'objects'`**

Top-level template name a widget's declared, live SpiriSynq objects are exposed under, e.g. `{{ objects.battery.voltage }}` for the alias `battery`.

`SpiriCamera.overlay.`**`VIEW_NAME = 'view'`**

Top-level template name for where the widget's objects are in the picture, for a widget that declares an `anchor` or `project`; see *world_placement()* and *Drawing on things in the world*.

class `SpiriCamera.overlay.`**`HudWidget`**(*synq_topic*, *synq_base_topic=None*, *synq_session=<factory>*, *synq_authoritive=False*, *synq_lazy_publish=False*, *synq_publish=True*, *synq_receive=True*, *synq_auto_start=True*, *synq_check_receive_types=True*, *synq_auto_rehydrate_on_missing_parent_timeout=5.0*, *synq_mtime=-1*, *synq_is_deleted=False*, *_synq_callbacks=<factory>*, *_synq_last_rehydrate_time=0.0*, *svg_template=''*, *anchor='top_left'*, *custom_anchor_x=0.0*, *custom_anchor_y=0.0*, *z_index=0.0*)

Bases: `SyncableObject`

One overlay element: an SVG template anchored on the frame.

Owned by whoever authors the overlay, not by the camera that draws it – a camera only mirrors widgets named in its `overlay_widgets` (see *OverlayMixin*), it never creates or publishes one.

Attributes

svg_template
: [str] MiniJinja source for the widget's SVG. May reference `{{ exif.* }}` for the current frame's tags, and `{{ objects.<alias>.<field> }}` for any other SpiriSynq object's latest field values, provided the same alias is declared with a `{# object: <alias> = <default topic> #}` comment somewhere in this same template; see *resolve_objects()*. The declared topic is only a default – a camera rendering this widget can bind the alias to a different object of its own choosing; see *OverlayMixin.overlay_widgets*.

anchor
: [str] This widget's *default* placement: one of the nine-point grid (`top_left` .. `bottom_right`) or `"custom"` to use *custom_anchor_x*/*custom_anchor_y* instead. A camera rendering this widget can override the placement for itself alone; see *OverlayMixin.overlay_widgets*.

custom_anchor_x
: [float] Percent (0-100) of the frame's width. Only used when *anchor* is `"custom"`, in which case it names the widget's top-left corner directly – unlike the named anchors, which use the widget's own rendered size to flush it against an edge or centre it.

custom_anchor_y
: [float] Percent (0-100) of the frame's height. See *custom_anchor_x*.

z_index
: [float] Paint order relative to this camera's other overlay widgets – higher paints on top, lower underneath. Widgets sharing a value keep the stable order they'd otherwise render in (insertion order of *OverlayMixin.overlay_widgets*). Ties with another widget's overlapping frame content (e.g. a bounding-box widget meant to sit under a HUD) are the usual reason to set this away from the default.

```
svg_template: str = ''

anchor: str = 'top_left'

custom_anchor_x: float = 0.0

custom_anchor_y: float = 0.0

z_index: float = 0.0
```

Parameters

- **synq_topic** (*str*)
- **synq_base_topic** (*str | None*)
- **synq_session** (*Session | None*)
- **synq_authoritive** (*bool*)
- **synq_lazy_publish** (*bool*)
- **synq_publish** (*bool*)
- **synq_receive** (*bool*)

- **synq_auto_start** (*bool*)
- **synq_check_receive_types** (*bool*)
- **synq_auto_rehydrate_on_missing_parent_timeout** (*float*)
- **synq_mtime** (*float*)
- **synq_is_deleted** (*bool*)
- **_synq_callbacks** (*dict*)
- **_synq_last_rehydrate_time** (*float*)
- **svg_template** (*str*)
- **anchor** (*str*)
- **custom_anchor_x** (*float*)
- **custom_anchor_y** (*float*)
- **z_index** (*float*)

exception SpiriCamera.overlay.**OverlayError**

Bases: ValueError

Raised when a widget's SVG cannot be templated or rasterized.

class SpiriCamera.overlay.**OverlayMixin**(*overlay_widgets=<factory>*, *overlay_client_render=False*)

Bases: object

Overlay support for a *CameraBase*.

The only synced field is *overlay_widgets*; every other piece of state this needs (mirrored widgets, mirrored data-source objects, per-widget error de-duplication, cached template-parse results) is plain instance state the host class sets up itself – see *Camera*'s constructor, which does the same for its own EXIF tag providers. This mixin has no `__post_init__` of its own and starts nothing by itself; the host class calls `_overlay_sync_widgets()` once after construction (to pick up a rehydrated or constructor-supplied `overlay_widgets` immediately) and `_render_overlays()` from its own capture pipeline.

Parameters

- **overlay_widgets** (*EventedDict*)
- **overlay_client_render** (*bool*)

overlay_widgets: EventedDict

`{widget_topic: usage}`, where `widget_topic` is a mirrored *HudWidget*'s absolute topic and `usage` is a plain dict of *this camera's* overrides for that widget – everything in it is optional:

- `"x"`, `"y"`: percent-of-frame position, overriding the widget's own `anchor`/`custom_anchor_x`/`custom_anchor_y` for this camera alone (same semantics as the widget's own `custom` anchor). Omitted -> use the widget's own placement, same as every other camera mirroring it.

- `"bindings": {alias: object_topic}`, overriding which object a widget's declared alias resolves to, for this camera alone. Omitted, or missing a given alias -> use that alias's own `{# object: alias = default_topic #}` declaration.
- `"z_index"`: overrides the widget's own *`HudWidget.z_index`* for this camera alone – e.g. a bounding-box widget authored to sit underneath, but wanted on top for one particular camera. Omitted -> use the widget's own `z_index`, same as every other camera mirroring it.

A widget's mere presence as a key is what makes this camera render it – there is no separate list of topics. An `EventedDict`, not a plain `dict`, so SpiriSynq republishes the whole mapping on any mutation (add, remove, or edit a usage dict) the same way it already does for any other evented container field – see `SyncableObject._zenoh_publish_changes`. Live: editing this at runtime adds, drops, rebinds, or repositions widgets on the next frame, no restart needed.

`overlay_client_render: bool = False`

When set, *`read()`* skips `_render_overlays()` entirely – the raw frame is encoded unmodified, and a consumer (e.g. the NiceGUI UI) is expected to call *`render_overlays_for_client()`* itself and render the result in the browser instead. A plain synced field like any other – flippable live, no camera reconstruction needed.

`render_overlays_for_client`(*frame_width*, *frame_height*, *canvas_width=None*, *canvas_height=None*)

SVG markup positioning every resolvable widget by itself, for a browser to render natively instead of `_render_overlays()` rasterizing and compositing it server-side.

Companion to `_render_overlays()` for a camera with *`overlay_client_render`* set – same widget iteration, same live object values, same *`anchor_position()`* math (so placement matches the server-rendered path pixel for pixel), but each widget is measured (`measure_size()`) rather than rasterized, then positioned with an SVG `<g transform>` instead of being alpha-blended into a raster.

Parameters

frame_width, frame_height

[int] The frame size overlays are anchored against – typically `self.received_width`/`self.received_height`, i.e. the size of the image the browser is actually displaying. Always what templates see as `frame.width`/`frame.height`.

canvas_width, canvas_height

[int, optional] The coordinate space widgets are measured and anchored in, when that differs from the frame – e.g. the on-screen size of the displayed image in CSS pixels, so a widget drawn `18` units tall stays 18 screen pixels tall however large the frame is. Omitted -> the frame size, so widgets scale with the image exactly as they do when burned in server-side.

Returns

str

Zero or more `<g transform="translate(...)">...</g>` elements, one per resolvable widget, meant to be wrapped by the caller in a `<svg viewBox="0 0 {canvas_width} {canvas_height}" width="100%" height="100%" preserveAspectRatio="xMidYMid meet">` – the explicit size and "meet" matter whenever the dis-

play box's aspect ratio can differ from the frame's (e.g. a resizable viewport), so this letterboxes the same way the `<img>` does rather than stretching non-uniformly to fill the box. Empty when `overlay_widgets` is empty or no widget currently resolves – same no-op contract as `_render_overlays()`.

Parameters

- **frame_width** (*int*)
- **frame_height** (*int*)
- **canvas_width** (*int | None*)
- **canvas_height** (*int | None*)

Return type

str

class SpiriCamera.overlay.**WorldPlacement**(*anchor=None*, *each=None*, *project=()*)

Bases: `object`

How a widget is drawn by where its objects are in the world.

What *`world_placement()`* read from a template's `anchor` and `project` declarations; see *Drawing on things in the world*.

Attributes

anchor

[str | None] The alias the widget is centred on, `{# anchor: <alias> #}`, or whose linked objects it is drawn on, one copy each, with *`each`*. `None` for a widget placed on the frame.

each

[str | None] The field of *`anchor`*'s object holding the links, `{# anchor: each <alias>.<field> #}`.

project

[tuple[str, …]] Aliases whose pixels the widget is given, as `{{ view.<alias> }}`, `{# project: <alias>, ... #}`.

anchor: str | None = None

each: str | None = None

project: tuple[str, ...] = ()

property aliases: tuple[str, ...]

Every alias this placement needs mirrored.

Parameters

- **anchor** (*str | None*)
- **each** (*str | None*)

- **project** (*tuple[str, ...]*)

SpiriCamera.overlay.**anchor_position**(*anchor*, *custom_x*, *custom_y*, *frame_width*, *frame_height*, *widget_width*, *widget_height*)

Pixel position for a widget's top-left corner.

Parameters

anchor
[str] A *HudWidget.anchor* value.

custom_x, custom_y
[float] *HudWidget.custom_anchor_x*/_y, used only when `anchor == "custom"`.

frame_width, frame_height
[int] Size of the frame the widget is being composited onto.

widget_width, widget_height
[int] Size of the rasterized widget, i.e. *rasterize()*'s output shape.

Returns

tuple[int, int]
`(x, y)` of the widget's top-left corner, in frame pixels. Not clamped to the frame – *composite()* clips it.

Parameters

- **anchor** (*str*)
- **custom_x** (*float*)
- **custom_y** (*float*)
- **frame_width** (*int*)
- **frame_height** (*int*)
- **widget_width** (*int*)
- **widget_height** (*int*)

Return type
tuple[int, int]

SpiriCamera.overlay.**camera_metrics_widget**(*camera*, *, *anchor='bottom_left'*)

Build and publish a widget showing a camera's own live stats.

A ready-made overlay rather than a hand-authored one: baking a camera's actual resolution, framerate, and quality onto its own picture is common enough – a debugging HUD, a sanity check that frames are really arriving at the rate they should – to not be re-typed as SVG each time.

`width`/`height`/`framerate` come from `{{ frame.* }}`, injected directly by `OverlayMixin._render_overlays()` rather than read as an object field: `received_width`/`received_height`/

`received_framerate` are deliberately never published as fields of their own (see `CameraBase.synq_skip_sync`), so there is no live object this widget – or any widget – could declare to reach them; see the module docstring for why `frame` exists as its own namespace for exactly this. `quality` is read *live* off `camera` itself instead, exactly like any other overlay widget's `objects.<alias>.<field>` values – `camera` is consulted only here, at build time, to learn its own topic, which becomes the `cam` alias's *default* binding (see the module docstring): no separate wiring step is needed, `cam.overlay_widgets[widget_topic] = {}` is enough to mirror this widget and have it resolve `cam` to `camera` itself.

The second line reads `{{ exif.timestamp }}` – the frame's own capture time, unlike the render-time values above – with a MiniJinja `default` filter so `Camera.exif_enabled = False` (which leaves `exif_tags` empty) renders `"untagged"` instead of failing the whole widget's render under this module's strict-undefined setting.

Parameters

camera
: [SpiriCamera.camera.Camera] The camera to describe.

anchor
: [str] Where to anchor the widget; see *`HudWidget.anchor`*.

Returns

HudWidget
: A published, authoritative widget under its own topic (`f"{camera.synq_topic}_metrics"`). The caller owns it, and should `close()` it when done – typically alongside `camera` itself.

Parameters

- **`camera`** (`Camera`)
- **`anchor`** (*`str`*)

Return type
HudWidget

`SpiriCamera.overlay.`**`composite`**(*frame*, *rgba*, *x*, *y*)

Alpha-blend a rasterized widget onto a copy of a BGR frame.

Parameters

frame
: [np.ndarray] A `(height, width, 3)` BGR frame, as read from a source.

rgba
: [np.ndarray] A widget raster from *`rasterize()`*.

x, y
: [int] Top-left corner to place `rgba` at, from *`anchor_position()`*. May be negative, or place `rgba` partly or fully outside `frame`; the overlap is clipped.

Returns

np.ndarray
A new array – `frame` is never mutated in place, so a camera with no overlays configured is unaffected by this module, and `_encode()`'s cache-by-identity check still sees a stable object when overlays happen to render byte-identical output.

Parameters

- **`frame`** (*ndarray*)
- **`rgba`** (*ndarray*)
- **`x`** (*int*)
- **`y`** (*int*)

Return type
ndarray

`SpiriCamera.overlay.`**`declared_objects`**(*svg_template*)

Alias -> default SpiriSynq topic declarations in a widget's own template.

A widget declares every object it needs right where it uses it, as a comment MiniJinja already strips at render time:

```
{# object: battery = azrael/battery_monitor #}
```

Parameters

svg_template
[str] A *`HudWidget.svg_template`*.

Returns

dict[str, str]
`{alias: default_topic}`, one entry per declaration found. A template with none declares nothing.

Parameters
`svg_template` (*str*)

Return type
dict[str, str]

`SpiriCamera.overlay.`**`rasterize`**(*svg*, *frame_width=None*, *frame_height=None*)

Parse and rasterize an SVG, at its own natural pixel size or scaled to a percentage of the frame.

Never forces a size for an ordinarily-sized widget: its declared `width`/`height` or `viewBox` *is* the pixel size it is rendered and, later, composited at, unscaled. See the module docstring for why, and for the one exception this function does implement: a root `<svg>` whose `width` and `height` are both percentages is

scaled – preserving its own aspect ratio – to fit within that percentage of `frame_width`/`frame_height`, the same way CSS `object-fit: contain` would.

Parameters

svg
: [str] Rendered SVG source, as returned by *render_svg()*.

frame_width, frame_height
: [int, optional] Size of the frame this is about to be composited onto. Only consulted – and required – when `svg`'s root `width`/ `height` are both percentages; ignored otherwise.

Returns

np.ndarray
: An `(height, width, 4) uint8` array, straight (not premultiplied) RGBA, transparent where nothing was drawn.

Raises

OverlayError
: If the SVG could not be parsed, declares no size to render at, or uses percentage sizing with no frame size given to scale it against.

Parameters

- **svg** (*str*)
- **frame_width** (*int | None*)
- **frame_height** (*int | None*)

Return type
ndarray

`SpiriCamera.overlay.`**render_svg**(*svg_template*, ***, *objects*, *exif_tags*, *frame_info*, *view=None*, *item=None*)

Render a widget's template against live object field values, EXIF tags, and the actual frame being rendered onto.

Parameters

svg_template
: [str] A *HudWidget.svg_template*.

objects
: [Mapping[str, Mapping[str, object]]] `{alias: {field: value}}` for the aliases *resolve_objects()* found in use, exposed as `{{ objects.<alias>.<field> }}`. An alias the template reads but that is missing here renders as undefined, which MiniJinja raises on – see `Raises` below.

exif_tags
: [Mapping[str, str]] The current frame's tags, exposed as `{{ exif.* }}`.

frame_info
: [Mapping[str, object]] The current frame's actual `width`/`height`/`framerate`, exposed as `{{ frame.* }}`. See *`FRAME_NAME`* and the module docstring for why these are injected directly instead of being reached through `objects`.

view
: [Mapping[str, object], optional] Where the widget's objects are in the picture, exposed as `{{ view.* }}`: the anchor's *`View`* fields, or `{alias: fields or None}` for a projecting widget. Only for a widget with a *`world_placement()`*.

item
: [Mapping[str, object], optional] The linked object an `anchor: each` copy is drawn for, its fields exposed as `{{ item.* }}`.

Returns

str
: The rendered SVG source.

Raises

OverlayError
: If the template is malformed, or reads an alias with no value yet (e.g. an object that has not resolved, or a field it does not have). Either way this is one frame's problem, not a reason to stop rendering the widget forever – the caller is expected to retry on the next frame.

Parameters

- **`svg_template`** (*`str`*)
- **`objects`** (*`Mapping[str, Mapping[str, object]]`*)
- **`exif_tags`** (*`Mapping[str, str]`*)
- **`frame_info`** (*`Mapping[str, object]`*)
- **`view`** (*`Mapping[str, object] | None`*)
- **`item`** (*`Mapping[str, object] | None`*)

Return type

str

`SpiriCamera.overlay.`**`resolve_objects`**(*svg_template*)

Aliases a widget's template actually reads, checked against its own declarations.

Cross-checks two things read out of the same template text: every `{{ objects.<alias>.<field> }}` MiniJinja's own static analysis (`undeclared_variables_in_str`) finds the render body actually using, against every alias *`declared_objects()`* finds declared. This is the enforcement point the module docstring describes – a widget author gets a clear, immediate error for a typo'd or missing declaration, rather than a silently-undefined value only visible in the rendered picture.

Parameters

svg_template
: [str] A *HudWidget.svg_template*.

Returns

dict[str, str]
: `{alias: default_topic}`, restricted to aliases the template body actually reads – a declared-but-unused alias is not mirrored.

Raises

OverlayError
: If the template is malformed – as it is part way through an edit – or reads `objects.<alias>` for an alias with no matching declaration.

Parameters

svg_template (*str*)

Return type

dict[str, str]

`SpiriCamera.overlay.`**`tiger_widget`**(*camera*, ***, *size_percent=20.0*, *anchor='center_middle'*)

Build and publish a widget showing the classic Ghostscript tiger.

A ready-made demonstration of percentage sizing (see *rasterize()*): the vendored artwork is a fixed, unrelated 900x900 `viewBox`, but `size_percent` is baked into the widget's root `<svg width="{size_percent}%" height="{size_percent}%">` so it always renders at that fraction of the frame – shrinking or growing with the camera's resolution – rather than at some fixed pixel size that would look tiny on a 4K frame and oversized on a small one.

Parameters

camera
: [SpiriCamera.camera.Camera] The camera this widget's topic is namespaced under. The artwork itself does not depend on `camera` otherwise.

size_percent
: [float] Fraction of the frame, in each dimension, the tiger is scaled to fit within, preserving its own aspect ratio.

anchor
: [str] Where to anchor the widget; see *HudWidget.anchor*.

Returns

HudWidget
: A published, authoritative widget under its own topic (`f"{camera.synq_topic}_tiger"`). The caller owns it, and should `close()` it when done – typically alongside `camera` itself.

Parameters

- **camera** (Camera)
- **size_percent** (*float*)
- **anchor** (*str*)

Return type
HudWidget

SpiriCamera.overlay.**world_placement**(*svg_template*)

A widget's `anchor` and `project` declarations.

Parameters

svg_template
[str] A *HudWidget.svg_template*.

Returns

WorldPlacement
Empty for a widget that declares neither, the usual case.

Raises

OverlayError
For more than one anchor, an anchor and a projection in one widget, `each` without a field (or a field without `each`), or an alias with no `{# object: #}` declaration.

Parameters
svg_template (*str*)

Return type
WorldPlacement

6.2.4 SpiriCamera.projection

Where a point in the world lands in a camera's picture.

The overlay uses this to put a widget on an object in SpiriSynq's frame tree (see *Drawing on things in the world*), so a widget's template never does the geometry itself; it's plain functions on poses, usable on their own.

A camera's own frame is FRD, like the rest of the frame tree: `x` along the optical axis, out of the lens; `y` to the right of the picture; `z` down it. So with the camera's `Pose` (in ECEF, like every root-frame pose) and its *Intrinsics*, a point at `(x, y, z)` in the camera's frame is at pixel `(cx + fx * y / x, cy + fy * z / x)`. A camera on a gimbal with an identity mount looks along the gimbal's `x` axis, as a rangefinder on the same gimbal does.

The lens is the pinhole model, with no distortion: a wide lens draws a point near its edge some pixels off from where this puts it.

SpiriCamera.projection.**NEAR = 0.01**

anything closer is treated as behind the camera.

Type
Nearest a point is drawn, in metres along the optical axis

class SpiriCamera.projection.**View**(*x*, *y*, *r*, *range*, *edge*)

Bases: object

A point as a camera sees it.

Attributes

x, y
[float] Its pixel, in the frame *project()* was given. Off the frame when the point is in front of the camera but out of shot.

r
[float | None] The radius in pixels of the point's 90% accuracy (its Accuracy cylinder) as seen from here: the circle around (x, y) that holds the cylinder's outline. None when the point's accuracy is unknown, or when the cylinder reaches behind the camera. It is the point's own uncertainty, not the camera's: see *Drawing on things in the world* for what it does and doesn't mean.

range
[float] Metres from the camera to the point.

edge
[bool] Whether the point, or its circle, crosses the frame's edge.

x: **float**

y: **float**

r: **float** | **None**

range: **float**

edge: **bool**

visible(*width*, *height*)

Whether any of the point's circle is on a width x height frame.

Parameters

- **width** (*int*)
- **height** (*int*)

Return type
bool

Parameters

- **x** (*float*)

- **y** (*float*)
- **r** (*float | None*)
- **range** (*float*)
- **edge** (*bool*)

SpiriCamera.projection.**pixel**(*lens*, *local*)

A point in the camera's frame as a pixel of *lens*, or None behind it.

Parameters

- **lens** (Intrinsics)
- **local** (*Offset*)

Return type
tuple[float, float] | None

SpiriCamera.projection.**project**(*camera*, *lens*, *point*, *width*, *height*)

Where *point* is in a `width` x `height` frame from *camera*.

camera and *point* are root-frame (ECEF) poses, *lens* the camera's intrinsics at any size (scaled to the frame here). None when the point is behind the camera, or the camera's pose is a point with no orientation to look along.

Parameters

- **camera** (*Pose*)
- **lens** (Intrinsics)
- **point** (*Pose*)
- **width** (*int*)
- **height** (*int*)

Return type
View | None

SpiriCamera.projection.**to_camera**(*camera*, *point*)

An ECEF *point* in *camera*'s own frame, or None if *camera* has no axes.

Parameters

- **camera** (*Pose*)
- **point** (*Offset*)

Return type
Offset | None

6.2.5 SpiriCamera.sources

Source handler registry and resolution.

Handlers register themselves simply by being imported: the registry walks *SourceBase* subclasses and asks each one whether it recognises a given source string. Adding a new protocol means adding a module here — no dispatch table to update, and no protocol-specific knowledge anywhere but the handler itself.

class SpiriCamera.sources.**CaptureSettings**(*max_width=1920*, *max_height=1080*, *max_framerate=30*)

Bases: object

What the camera is asking the source to produce.

These are upper bounds, not exact dimensions: a source is free to deliver something smaller, and reports what it settled on through *SourceCapabilities*.

> **Note**
>
> Do not shorten these to `width` / `height` / `framerate`. The `max_` prefix is the contract: a source may answer a request for 1920x1080 with a 640x480 frame and still be correct. A bare `width` would read as "the width you will get", which is a promise no source makes. The names also match the *CameraBase* fields they are copied from, so the two stay greppable together.

Attributes

max_width
: [int] Largest acceptable frame width in pixels.

max_height
: [int] Highest requested frames per second.

max_framerate
: [int] Highest requested frames per second.

```
max_width: int = 1920

max_height: int = 1080

max_framerate: int = 30
```

Parameters

- **max_width** (*int*)
- **max_height** (*int*)
- **max_framerate** (*int*)

class SpiriCamera.sources.**Intrinsics**(*width*, *height*, *fx*, *fy*, *cx*, *cy*, *origin=''*)

Bases: object

A pinhole camera model, at the resolution it was measured at.

The model is in pixels of a `width` x `height` image. A frame of another size with the same aspect – a camera asked for a smaller `max_width`, say – uses *scaled()* to that size, rather than the numbers as they are.

Synced as one value, not field by field, so a peer never sees one report's focal length with another's size.

Attributes

width
[int] Image width the model is in, in pixels.

height
[int] Image height the model is in, in pixels.

fx
[float] Horizontal focal length, in pixels.

fy
[float] Vertical focal length, in pixels.

cx
[float] Principal point, horizontally, in pixels from the left edge.

cy
[float] Principal point, vertically, in pixels from the top edge.

origin
[str] Where it came from: `manual` (the camera's settings), `source` (the source reported it, e.g. a Gazebo sensor's `camera_info`) or `mavlink` (the camera's MAVLink component or autopilot, through SpiriMav). See *INTRINSICS_ORIGINS*.

width: **int**

height: **int**

fx: **float**

fy: **float**

cx: **float**

cy: **float**

origin: **str** = ''

classmethod from_fov(*width*, *height*, *hfov*, *vfov=0.0*, *origin=''*)

The model of a centred lens with a given field of view.

Parameters

width, height
[int] Image size to express the model in.

hfov
[float] Horizontal field of view, in degrees.

vfov
[float] Vertical field of view, in degrees. `0` assumes square pixels, so it follows from `hfov` and the image's aspect.

origin
[str] See the class attributes.

Parameters

- **width** (*int*)
- **height** (*int*)
- **hfov** (*float*)
- **vfov** (*float*)
- **origin** (*str*)

Return type
Intrinsics

property hfov: float

Horizontal field of view through the principal point, in degrees.

property vfov: float

Vertical field of view through the principal point, in degrees.

scaled(*width*, *height*)

The same lens, for an image resized to `width` x `height`.

Parameters

- **width** (*int*)
- **height** (*int*)

Return type
Intrinsics

Parameters

- **width** (*int*)
- **height** (*int*)
- **fx** (*float*)
- **fy** (*float*)

- **cx** (*float*)
- **cy** (*float*)
- **origin** (*str*)

class SpiriCamera.sources.**OpenCVSource**(*url*)

Bases: *SourceBase*

Base for sources backed by `cv2.VideoCapture`.

Subclasses supply the capture target via *capture_target()* and may extend *device_metadata()* to report vendor, model, and serial number.

abstractmethod capture_target()

Return the argument to hand to `cv2.VideoCapture`.

Returns

str | int
A device path, stream URL, or numeric camera index.

Return type
str | int

device_metadata()

Look up vendor, model, and serial number for the open device.

The default implementation reports nothing. Subclasses override when their transport exposes device identity.

Returns

dict[str, str]
Any of `vendor`, `model`, and `serial_number`.

Return type
dict[str, str]

capture_params()

Return extra `cv2.VideoCapture` open parameters.

The default implementation passes none. Subclasses override to set properties that only take effect when given at open time, such as FFmpeg's open and read timeouts.

Returns

list[int]
Alternating `cv2.CAP_PROP_*` ids and values.

Return type
list[int]

property is_open: bool
Whether the capture object is open.

open(*settings*)
Open the capture device and negotiate settings.

Parameters

settings
[CaptureSettings] Requested resolution and framerate.

Returns

SourceCapabilities
What the device actually negotiated.

Raises

SourceError
If the device cannot be opened.

Parameters
settings (CaptureSettings)

Return type
SourceCapabilities

close()
Release the capture device.

Return type
None

read(*settings*)
Grab a single BGR frame from the device.

Re-applies `settings` first if they differ from what is currently on the device, so a live change to `max_width` and friends takes effect on the next frame rather than only at the next `open()`. Most devices accept a resolution change mid-stream; one that does not simply keeps delivering its current size, which `CameraBase.received_width` and friends will still report correctly.

Parameters

settings
: [CaptureSettings] Requested capture settings.

Returns

np.ndarray | None
: The frame, or `None` if the device produced nothing.

Raises

SourceError
: If the source was never opened.

Parameters
: **settings** (CaptureSettings)

Return type
: *ndarray* | None

class SpiriCamera.sources.**SourceBase**(*url*)

Bases: ABC

Abstract base for camera source handlers.

A handler is responsible for its own protocol end to end: deciding whether it recognises a source string, parsing it, opening the underlying device, and producing raw BGR frames. Registration is implicit — *SpiriCamera.sources* discovers every concrete subclass that has been imported.

Subclasses declare the schemes they own via *schemes* and implement *open()*, *close()*, *read()*, and *is_open*. A handler that accepts schemeless strings (such as a V4L2 device path) also overrides *handles()*.

Attributes

schemes
: [ClassVar[tuple[str, …]]] URL schemes this handler claims, without the `://` separator.

schemes: ClassVar[tuple[str, ...]] = ()

classmethod handles(*url*)

Report whether this handler recognises a source URL.

The default implementation matches *schemes*. Override to accept schemeless forms.

Parameters

url
: [SourceURL] The parsed source string.

Returns

bool
True if this handler can serve the URL.

Parameters
url (SourceURL)

Return type
bool

classmethod from_url(*url*)

Instantiate this handler for a source URL it has claimed.

Override when a handler needs to validate or normalise the target before construction.

Parameters

url
[SourceURL] The parsed source string.

Returns

Self
A handler instance, not yet opened.

Raises

SourceError
If the target is malformed for this scheme.

Parameters
url (SourceURL)

Return type
Self

property scheme: str

Normalised scheme name, filled in for schemeless sources.

property target: str

device path, stream URL, or image name.

Type
Scheme-specific target

describe()

Summarise this source for the camera's synced state.

Returns

SourceInfo
A network-transparent description of this handler.

Return type
SourceInfo

abstract property is_open: bool

Whether the underlying device is currently open.

abstractmethod open(*settings*)

Open the underlying device and negotiate capture settings.

Calling *open()* on an already-open source is a no-op that re-reports the current capabilities.

Parameters

settings
[CaptureSettings] The resolution and framerate the camera is asking for.

Returns

SourceCapabilities
What the source will actually deliver.

Raises

SourceError
If the device cannot be opened.

Parameters
settings (CaptureSettings)

Return type
SourceCapabilities

abstractmethod close()

Release the underlying device.

Closing an already-closed source is a no-op.

Return type
None

abstractmethod read(*settings*)

Produce a single frame.

Handlers return raw pixels; JPEG encoding and publication belong to the camera layer.

Parameters

settings
: [CaptureSettings] The resolution and framerate currently requested. Sources that render on demand should honour this on every call so that live resolution changes take effect.

Returns

np.ndarray | None
: A BGR frame, or `None` when no new frame is available. Returning the *same* array object on consecutive calls tells the camera the frame is unchanged and lets it reuse its previous encode.

Raises

SourceError
: If the source has failed in a way a retry will not fix.

Parameters

settings (CaptureSettings)

Return type

ndarray | None

frame_tags()

Tags that arrived with the frame *read()* last returned.

For sources whose frames were already tagged before they got here – another camera's output, say – so the metadata survives being decoded and re-encoded. The camera merges these under its own tags; see *exif_tags_for_frame()*.

Returns

dict[str, str]
: The inherited tags. The default reports none.

Return type

dict[str, str]

frame_intrinsics()

The lens of the frame *read()* last returned, if it changes.

For sources whose lens can change while open – another camera's output, whose lens zooms – so the camera follows it rather than keeping the one *open()* reported.

Returns

Intrinsics | None
The lens now, or `None` to keep the last one reported. The default keeps what *`open()`* said.

Return type
Intrinsics | None

Parameters
`url` (`SourceURL`)

class SpiriCamera.sources.**SourceCapabilities**(*max_supported_width=None*, *max_supported_height=None*, *max_supported_framerate=None*, *vendor=''*, *model=''*, *serial_number=''*, *intrinsics=None*, *frame=''*)

Bases: `object`

What a source reports it can provide, once opened.

Note

These keep the `max_supported_` prefix for the same reason *`CaptureSettings`* keeps `max_`: they are ceilings the device advertises, not a guarantee about any particular frame, and they are copied verbatim onto the matching *`CameraBase`* fields.

Attributes

max_supported_width
[int | None] Largest width the source can deliver, or `None` if it renders at whatever size it is asked for.

max_supported_height
[int | None] Largest height the source can deliver, or `None` if it is resolution independent.

max_supported_framerate
[int | None] Highest framerate the source can deliver, or `None` if unconstrained.

vendor
[str] Device vendor, when the source can determine one.

model
[str] Device model, when the source can determine one.

serial_number
[str] Device serial number, when the source can determine one.

intrinsics
[Intrinsics | None] The lens, when the source knows it – a simulated camera does, exactly; most devices don't say. Its `origin` is filled in by the camera.

frame

[str] The SpiriSynq topic of the `Position` the frames are seen from, when they are some other object's view: a relayed camera's. The camera follows its pose unless it has been placed itself. Empty for a source that is the camera.

max_supported_width: *int | None* = None

max_supported_height: *int | None* = None

max_supported_framerate: *int | None* = None

vendor: *str* = ''

model: *str* = ''

serial_number: *str* = ''

intrinsics: *Intrinsics | None* = None

frame: *str* = ''

Parameters

- **max_supported_width** (*int | None*)
- **max_supported_height** (*int | None*)
- **max_supported_framerate** (*int | None*)
- **vendor** (*str*)
- **model** (*str*)
- **serial_number** (*str*)
- **intrinsics** (*Intrinsics | None*)
- **frame** (*str*)

exception SpiriCamera.sources.**SourceError**

Bases: `Exception`

Base class for source resolution, open, and read failures.

class SpiriCamera.sources.**SourceInfo**(*url=''*, *scheme=''*, *target=''*, *handler=''*, *error=''*)

Bases: `SubSyncableDataclass`

Network-transparent description of a camera's resolved source.

This is the source-shaped half of the camera's synced state: it travels over SpiriSynq so a remote peer can see what a camera is pointed at without having to parse the source string itself.

Attributes

url
: [str] The source string this description was resolved from.

scheme
: [str] Normalised scheme (`v4l`, `rtsp`, `testimage`, …).

target
: [str] Scheme-specific target: device path, stream URL, or image name.

handler
: [str] Name of the handler class that claimed the source.

error
: [str] Empty when the source resolved cleanly, otherwise the reason it could not be resolved.

```
url: str = ''
```

```
scheme: str = ''
```

```
target: str = ''
```

```
handler: str = ''
```

```
error: str = ''
```

classmethod unresolved(*url*, *error*)

Build a description for a source string that could not be resolved.

Parameters

url
: [str] The offending source string.

error
: [str] Human-readable reason the source was rejected.

Returns

SourceInfo
: A description carrying only `url` and `error`.

Parameters

- **url** (*str*)
- **error** (*str*)

Return type
: SourceInfo

update(*other*)

Copy another description's fields into this one, field by field.

Updating in place keeps the object identity stable, so UI bindings and sync subscriptions attached to this instance survive a source change, and only the fields that actually changed are published.

Parameters

other

[SourceInfo] The description to copy from.

Parameters

other (SourceInfo)

Return type

None

Parameters

- **url** (*str*)
- **scheme** (*str*)
- **target** (*str*)
- **handler** (*str*)
- **error** (*str*)

class SpiriCamera.sources.**SourceURL**(*raw*, *scheme*, *target*)

Bases: object

A source string split into its scheme and target.

Attributes

raw

[str] The original (stripped) source string.

scheme

[str] Lowercased scheme, or `""` for schemeless strings such as `/dev/video0` or a bare camera index.

target

[str] Everything after `://`, or the whole string when schemeless.

raw: str

scheme: str

target: str

classmethod parse(*source_str*)

Split a source string into scheme and target.

Parameters

source_str
[str] The source string to parse.

Returns

SourceURL
The split representation.

Raises

SourceError
If the source string is empty or whitespace only.

Parameters
source_str (*str*)

Return type
SourceURL

Parameters

- **raw** (*str*)
- **scheme** (*str*)
- **target** (*str*)

exception SpiriCamera.sources.**UnknownSourceError**

Bases: *SourceError*

Raised when no registered handler recognises a source string.

SpiriCamera.sources.**describe_source**(*source_str*)

Resolve a source string and describe it without opening anything.

Parameters

source_str
[str] The source string to inspect.

Returns

SourceInfo
A network-transparent description of the resolved source.

Raises

SourceError
If the source string cannot be resolved.

Parameters

source_str (*str*)

Return type

SourceInfo

SpiriCamera.sources.**known_schemes**()

List the URL schemes the registered handlers claim.

Returns

list[str]
Sorted scheme names, formatted as `scheme://`.

Return type

list[str]

SpiriCamera.sources.**registered_sources**()

List every concrete source handler currently importable.

Returns

list[type[SourceBase]]
Concrete *SourceBase* subclasses, depth-first.

Return type

list[type[SourceBase]]

SpiriCamera.sources.**resolve_source**(*source_str*)

Find and instantiate the handler for a source string.

Handlers that claim the URL's explicit scheme win over handlers that match schemeless strings, so `v4l://0` and `/dev/video0` both land on the V4L2 handler without either rule shadowing the other.

Parameters

source_str
[str] Source string such as `/dev/video0`, `rtsp://host/stream`, or `testimage://pm5544`.

Returns

SourceBase
A handler instance, not yet opened.

Raises

SourceError
If the source string is empty or malformed for its scheme.

UnknownSourceError
If no registered handler recognises the source string.

Parameters
source_str (*str*)

Return type
SourceBase

6.2.6 SpiriCamera.webrtc

WebRTC plumbing shared by every signaling path.

Two very different callers need to turn an SDP offer into a live `RTCPeerConnection` publishing a *Camera*'s frames:

- *SpiriCamera.camera.Camera.webrtc_offer()*, a `@remote_method` invoked synchronously from a zenoh queryable callback thread with no event loop of its own.
- *SpiriCamera.whep*'s `POST /whep` route, an `async def` running on uvicorn's event loop.

Both end up calling `_negotiate()` on the *same* background event loop (*get_loop()*), which this module starts once, on first use, and never stops. That loop is what keeps every peer connection's ICE/SRTP/media machinery alive after the call that created it returns – there is nothing else left holding a reference to it once a zenoh RPC handler hands back its reply. `negotiate_sync` blocks the calling thread for the result; `negotiate_async` is the awaitable equivalent for a caller that already has its own loop. Every session, however it was opened, lives in the one `_SESSIONS` registry, so closing it works the same way regardless of which door it came in.

Frame tags travel over an `RTCDataChannel`, but only one way round: the *viewer* must create the channel (labelled *TAGS_CHANNEL_LABEL*) before sending its offer, because a single-shot SDP answer can only describe `m=` sections the offer already listed – an answerer cannot introduce a new one without a second round of negotiation. A plain WHEP player that only asks for video gets video only; a SpiriCamera-aware client that opens the tags channel in its offer gets one JSON message per frame, `{"timestamp": ..., "tags": {...}}`, read straight back out of that frame's own EXIF via *SpiriCamera.exif.extract()* – the same tags *Camera.exif_tags_for_frame* already baked in, not a second copy of them.

Every published session adapts its send rate to the link on its own, from the packet loss the viewer reports – see `_RateController`.

SpiriCamera.webrtc.**TAGS_CHANNEL_LABEL = 'spiricamera-tags'**

Label a viewer's offer must use for its data channel to receive tags.

SpiriCamera.webrtc.**close_peer_sync**(*pc*, *timeout=5.0*)

Close a peer connection created by *pull_sync()*.

Parameters

- **pc** (*RTCPeerConnection*)

- **timeout** (*float*)

Return type
None

async SpiriCamera.webrtc.**close_session_async**(*session_id*, *timeout=5.0*)

Awaitable equivalent of *close_session_sync()*.

Parameters

- **session_id** (*str*)
- **timeout** (*float*)

Return type
bool

SpiriCamera.webrtc.**close_session_sync**(*session_id*, *timeout=5.0*)

Close a session by id. Returns `False` if it was already gone.

Parameters

- **session_id** (*str*)
- **timeout** (*float*)

Return type
bool

SpiriCamera.webrtc.**get_loop**()

Return the shared background event loop, starting it if needed.

Every `RTCPeerConnection` this module creates lives on this loop for its entire life, so it keeps running forever in a daemon thread rather than being torn down between calls.

Return type
AbstractEventLoop

async SpiriCamera.webrtc.**negotiate_async**(*camera*, *sdp*, *type_='offer'*, *timeout=10.0*)

Awaitable entry point, for a caller already running one.

Parameters

- **camera** (*object*)
- **sdp** (*str*)
- **type_** (*str*)
- **timeout** (*float*)

Return type
tuple[str, str, str]

SpiriCamera.webrtc.**negotiate_sync**(*camera*, *sdp*, *type_='offer'*, *timeout=10.0*)

Synchronous entry point, for a caller with no event loop of its own.

Parameters

- **camera** (*object*)
- **sdp** (*str*)
- **type_** (*str*)
- **timeout** (*float*)

Return type
tuple[str, str, str]

SpiriCamera.webrtc.**pull_sync**(*url*, *timeout=15.0*)

Synchronous entry point for `_whep_pull()`.

Parameters

- **url** (*str*)
- **timeout** (*float*)

Return type
RTCPeerConnection

6.2.7 SpiriCamera.whep

WHEP (WebRTC-HTTP Egress Protocol) exposure for a single camera.

This is the HTTP door onto the same session machinery *SpiriCamera.camera.Camera.webrtc_offer()* uses over zenoh – both call straight into *SpiriCamera.webrtc*, so a viewer that came in over HTTP and one that came in over SpiriSynq get the same video track and tags data channel.

Deliberately minimal: one camera per app (*create_whep_app()* takes a callable returning it, matching the "expose *a* topic over http" ask rather than a multi-camera router), and no `PATCH` route for trickle ICE – *SpiriCamera.webrtc* waits for ICE gathering to finish before answering, so there is nothing left to trickle. A future need for faster connection setup through restrictive NATs is the reason to revisit that, not anything this module is missing today.

Run standalone with `SpiriCamera whep-serve <source>` (see *SpiriCamera.cli*), or mount `create_whep_app(...)` into any other FastAPI app.

SpiriCamera.whep.**SDP_MEDIA_TYPE = 'application/sdp'**

Media type both the offer and the answer are carried in, per WHEP.

SpiriCamera.whep.**create_whep_app**(*get_camera*)

Build a WHEP app publishing whatever `get_camera()` returns.

Parameters

get_camera
[Callable[[], Camera]] Called on every request, so the camera it returns can be swapped or reconfigured out from under a long-lived app.

Returns

FastAPI
An app with `POST /whep` and `DELETE /whep/{session_id}`.

Parameters

get_camera (*Callable[[], Camera]*)

Return type

fastapi.FastAPI

6.2.8 SpiriCamera.ui

NiceGUI test UI for SpiriCamera.

Frames are served over a plain HTTP route and pulled by an `ui.interactive_image`, rather than pushed as base64 data URLs over the websocket. Two reasons, both of which showed up as visible artefacts:

- `ui.image` wraps Quasar's `QImg`, which cross-fades every time its source changes. At video framerates that fade never finishes, so the picture looks permanently dimmed and pulses at a constant rate. `ui.interactive_image` has no transition.
- A data URL is pushed to every client for every frame whether or not the browser can keep up. Pulling instead lets the component skip frames it cannot load in time, which is what makes it adapt to the available bandwidth.

See https://nicegui.io/documentation/interactive_image and NiceGUI's `examples/opencv_webcam`.

Every control on this page binds two-way with `bind_value`, including the fields the camera normally fills in for itself. This is deliberate: no SpiriSynq property is truly read-only, a remote peer can write any of them already, and this page exists to debug that. Use `bind_value`, not `bind_value_from`, unless a field genuinely has nowhere to write back to. Two things here qualify: the bandwidth meter, which is a measurement taken at the HTTP route, and the frame tags, which are read out of a frame that has already been encoded and sent. Both have a writable counterpart nearby — the camera's own settings, and Extra Tags.

`SpiriCamera.ui.`**`FRAME_ROUTE = '/camera/frame'`**

Route the browser pulls frames from.

`SpiriCamera.ui.`**`add_metrics_widget`**(*cam*)

Publish a *camera_metrics_widget()* for `cam` and add it to its own `overlay_widgets`.

Parameters

cam
[Camera] The camera to describe and attach the widget to.

Returns

HudWidget
The published widget, already added to `cam.overlay_widgets`.

Parameters

cam (Camera)

Return type
HudWidget

SpiriCamera.ui.**add_tiger_widget**(*cam*)

Publish a *tiger_widget()* for cam and add it to its own overlay_widgets.

A worked example of percentage sizing (see *rasterize()*) alongside the pixel-sized camera_metrics_widget example next to it.

Parameters

cam
[Camera] The camera to attach the widget to.

Returns

HudWidget
The published widget, already added to cam.overlay_widgets.

Parameters
cam (Camera)

Return type
HudWidget

SpiriCamera.ui.**build_page**()

Build the camera test UI page.

SpiriCamera.ui.**create_overlay_widget**(*cam*, *name*)

Publish a new, editable HudWidget and add it to cam.overlay_widgets.

Parameters

cam
[Camera] The camera to attach the new widget to.

name
[str] A short name for the widget, used to build its topic. Slugified the same way *topic_for_source()* slugifies a source string, so anything typeable works.

Returns

HudWidget
The published widget, already added to cam.overlay_widgets.

Parameters

- **cam** (Camera)
- **name** (*str*)

Return type
HudWidget

SpiriCamera.ui.**discover_cameras**(*cam*)

List Camera objects other nodes are currently advertising.

Parameters

cam
[Camera] Any camera on the session to query from – discovery is a property of the SpiriSynq session, not of a particular camera.

Returns

list[dict]
One metadata dict per discovered camera, each with at least `topic` and `authoritative_node`. Only authoritative cameras ever answer this query, so a mirror never lists itself.

Parameters
cam (Camera)

Return type
list[dict]

SpiriCamera.ui.**discover_widgets**(*cam*)

List HudWidget objects other nodes are currently advertising.

Parameters

cam
[Camera] Any camera on the session to query from; see *discover_cameras()*.

Returns

list[dict]
One metadata dict per discovered widget, each with at least `topic`.

Parameters
cam (Camera)

Return type
list[dict]

SpiriCamera.ui.**get_camera**()

Return the process-wide camera, creating it on first use.

One camera per process, not one per page: a camera is a device, and every instance declares its own SpiriSynq topic.

Returns

Camera
The shared camera instance.

Return type
Camera

SpiriCamera.ui.**remove_overlay_widget**(*cam*, *topic*)

Detach `topic` from `cam.overlay_widgets`, closing it if this page is the one that created it.

Parameters

cam
[Camera] The camera to detach the widget from.

topic
[str] Absolute topic of the widget to remove, as it appears in `cam.overlay_widgets`.

Parameters

- **cam** (Camera)
- **topic** (*str*)

Return type
None

SpiriCamera.ui.**serve_frame**()

Serve the most recent frame as an image response.

Returns

Response
The latest encoded frame, or a placeholder if none exists yet.

Return type
fastapi.Response

SpiriCamera.ui.**set_authoritive**(*source_str=''*)

Switch to running a device ourselves, replacing any mirror.

Parameters

source_str
[str] Source to open. Defaults to the test image when switching out of mirror mode, which never has a local source string of its own to fall back to.

Returns

Camera
The new authoritative camera.

Parameters
source_str (*str*)

Return type
Camera

SpiriCamera.ui.**set_mirror**(*topic=''*)

Switch to mirroring another node's camera, replacing any device.

Parameters

topic
[str] Absolute SpiriSynq topic of the camera to mirror, as reported by *discover_cameras()*. Empty to drop into mirror mode without yet picking one, e.g. to browse what is available.

Returns

Camera
The new, non-authoritative camera. A stopped device with nothing to show until a topic is given: its fields sit at their defaults rather than mirroring anything.

Parameters
topic (*str*)

Return type
Camera

6.2.9 SpiriCamera.web

Web UI entry point for SpiriCamera (optional SpiriConfig plugin).

6.2.10 SpiriCamera.cli

CLI interface for SpiriCamera.

SpiriCamera.cli.**main_callback**(*ctx*, *verbose=False*)

Enter callback for SpiriCamera.

Parameters

- **ctx** (*Context*)
- **verbose** (*Annotated[bool, <typer.models.OptionInfo object at 0x7c466067c7d0>]*)

SpiriCamera.cli.**version**()

Show version information.

Return type
None

SpiriCamera.cli.**sources**()

List the registered source handlers and the schemes they claim.

Return type
None

SpiriCamera.cli.**validate**(*source=''*)

Validate a camera source, then report what the device can do.

Parameters
source (*Annotated[str, <typer.models.ArgumentInfo object at 0x7c46606e74d0>]*)

Return type
None

SpiriCamera.cli.**run**(*source=''*)

Capture continuously and publish frames to SpiriSynq until interrupted.

Parameters
source (*Annotated[str, <typer.models.ArgumentInfo object at 0x7c46606e74d0>]*)

Return type
None

SpiriCamera.cli.**capture**(*source=''*, *frames=1*, *quality=80*, *output=''*)

Capture a fixed number of frames and write them out.

Parameters

- **source** (*Annotated[str, <typer.models.ArgumentInfo object at 0x7c46606e74d0>]*)
- **frames** (*Annotated[int, <typer.models.OptionInfo object at 0x7c466067c2d0>]*)
- **quality** (*Annotated[int, <typer.models.OptionInfo object at 0x7c466067c410>]*)
- **output** (*Annotated[str, <typer.models.OptionInfo object at 0x7c466067c550>]*)

Return type
None

SpiriCamera.cli.**whep_serve**(*source=''*, *host='0.0.0.0'*, *port=8889*)

Run a camera and expose it for WHEP playback over HTTP.

Parameters

- **source** (*Annotated[str, <typer.models.ArgumentInfo object at 0x7c46606e74d0>]*)
- **host** (*Annotated[str, <typer.models.OptionInfo object at 0x7c466067cb90>]*)
- **port** (*Annotated[int, <typer.models.OptionInfo object at 0x7c466067ccd0>]*)

Return type
None

6.2.11 SpiriCamera.main

Main entry point for SpiriCamera.

class SpiriCamera.main.**Settings**(*_case_sensitive=None*, *_nested_model_default_partial_update=None*, *_env_prefix=None*, *_env_prefix_target=None*, *_env_file=PosixPath('.')*, *_env_file_encoding=None*, *_env_file_depth=None*, *_env_ignore_empty=None*, *_env_nested_delimiter=None*, *_env_nested_max_split=None*, *_env_parse_none_str=None*, *_env_parse_enums=None*, *_cli_prog_name=None*, *_cli_parse_args=None*, *_cli_settings_source=None*, *_cli_parse_none_str=None*, *_cli_hide_none_type=None*, *_cli_avoid_json=None*, *_cli_enforce_required=None*, *_cli_use_class_docs_for_groups=None*, *_cli_show_env_vars=None*, *_cli_exit_on_error=None*, *_cli_prefix=None*, *_cli_flag_prefix_char=None*, *_cli_implicit_flags=None*, *_cli_ignore_unknown_args=None*, *_cli_kebab_case=None*, *_cli_shortcuts=None*, *_secrets_dir=None*, *_build_sources=None*, ***, *app_name='SpiriCamera'*, *version='0.1.0'*, *environment='development'*, *debug=False*, *log_level='INFO'*, *source=''*, *topic=''*, *quality=80*, *frame_width=1920*, *frame_height=1080*, *framerate=30*, *hfov=0.0*, *vfov=0.0*, *fx=0.0*, *fy=0.0*, *cx=0.0*, *cy=0.0*, *calibration_width=0*, *calibration_height=0*, *relative_to=''*)

Bases: BaseSettings

Settings for SpiriCamera, loaded from environment variables.

Environment variables are prefixed with `SPIRICAMERA_`. Example:

```
export SPIRICAMERA_SOURCE="rtsp://admin:pass@192.168.1.100:554/stream"
export SPIRICAMERA_QUALITY=85
export SPIRICAMERA_FRAME_WIDTH=1280
export SPIRICAMERA_FRAME_HEIGHT=720
export SPIRICAMERA_FRAMERATE=30
```

Attributes

app_name
[str] Application name displayed in logs.

version
[str] Current package version.

environment
[str] Runtime environment (`development`, `production`, etc.).

debug
[bool] Enable verbose debugging output.

log_level
[str] Logging level (`DEBUG`, `INFO`, `WARNING`, `ERROR`).

source
[str] Camera source string (e.g. `/dev/video0`, `rtsp://...`).

quality
[int] JPEG encoding quality for captured frames [1-100].

frame_width
[int] User-requested capture width (default 1920).

frame_height
[int] User-requested capture height (default 1080).

framerate
[int] User-requested capture framerate (default 30).

hfov, vfov
[float] The lens's field of view in degrees, e.g. from its datasheet; `0` is unset. `vfov` unset assumes square pixels.

fx, fy, cx, cy
[float] A calibrated lens instead, in pixels of a `calibration_width` x `calibration_height` image (OpenCV's camera matrix). Wins over `hfov`/`vfov`; `cx`/`cy` unset are the image centre.

calibration_width, calibration_height
[int] The image size `fx`… were measured at.

relative_to
[str] The SpiriSynq topic of what the camera is mounted on, a gimbal's (`mu-1/robot_gimbal1`), so the camera has a pose and looks where it does. Mounted with no offset and the same axes; set `mount` over SpiriSynq for any other mounting. Empty leaves the camera unplaced, unless its source places it (a `spirisynq://` relay follows the camera it relays).

model_config = {'arbitrary_types_allowed': True, 'case_sensitive': False, 'cli_avoid_json': False, 'cli_enforce_required': False, 'cli_exit_on_error': True, 'cli_flag_prefix_char': '-', 'cli_hide_none_type': False, 'cli_ignore_unknown_args': False, 'cli_implicit_flags': False, 'cli_kebab_case': False, 'cli_parse_args': None, 'cli_parse_none_str': None, 'cli_prefix': '', 'cli_prog_name': None, 'cli_shortcuts': None, 'cli_show_env_vars': False, 'cli_use_class_docs_for_groups': False, 'enable_decoding': True, 'env_file': '.env', 'env_file_depth': 0, 'env_file_encoding': None, 'env_ignore_empty': False, 'env_nested_delimiter': None, 'env_nested_max_split': None, 'env_parse_enums': None, 'env_parse_none_str': None, 'env_prefix': 'SPIRICAMERA_', 'env_prefix_target': 'variable', 'extra': 'ignore', 'json_file': None, 'json_file_encoding': None, 'nested_model_default_partial_update': False, 'protected_namespaces': ('model_validate', 'model_dump', 'settings_cached', 'settings_clear_cache', 'settings_customise_sources'), 'secrets_case_sensitive': None, 'secrets_dir': None, 'secrets_dir_max_size': None, 'secrets_dir_missing': None, 'secrets_nested_delimiter': None, 'secrets_nested_subdir': None, 'secrets_prefix': None, 'toml_file': None, 'validate_default': True, 'yaml_config_section': None, 'yaml_file': None, 'yaml_file_encoding': None}

Configuration for the model, should be a dictionary conforming to [*ConfigDict*][pydantic.config.ConfigDict].

app_name: str

version: str

environment: str

debug: bool

log_level: str

source: str

topic: str

quality: int

frame_width: int

frame_height: int

framerate: int

hfov: float

vfov: float

fx: float

fy: float

cx: float

cy: float

calibration_width: int

calibration_height: int

relative_to: str

intrinsics()

The configured lens as `Camera.set_intrinsics` arguments, if any.

Return type
dict[str, float] | None

Parameters

- **_case_sensitive** (*bool | None*)
- **_nested_model_default_partial_update** (*bool | None*)
- **_env_prefix** (*str | None*)
- **_env_prefix_target** (*EnvPrefixTarget | None*)
- **_env_file** (*DotenvType | None*)
- **_env_file_encoding** (*str | None*)
- **_env_file_depth** (*int | None*)
- **_env_ignore_empty** (*bool | None*)
- **_env_nested_delimiter** (*str | None*)
- **_env_nested_max_split** (*int | None*)
- **_env_parse_none_str** (*str | None*)
- **_env_parse_enums** (*bool | None*)
- **_cli_prog_name** (*str | None*)
- **_cli_parse_args** (*bool | list[str] | tuple[str, ...] | None*)
- **_cli_settings_source** (*CliSettingsSource[Any] | None*)
- **_cli_parse_none_str** (*str | None*)
- **_cli_hide_none_type** (*bool | None*)
- **_cli_avoid_json** (*bool | None*)
- **_cli_enforce_required** (*bool | None*)

- **_cli_use_class_docs_for_groups** (*bool | None*)
- **_cli_show_env_vars** (*bool | None*)
- **_cli_exit_on_error** (*bool | None*)
- **_cli_prefix** (*str | None*)
- **_cli_flag_prefix_char** (*str | None*)
- **_cli_implicit_flags** (*bool | Literal['dual', 'toggle'] | None*)
- **_cli_ignore_unknown_args** (*bool | None*)
- **_cli_kebab_case** (*bool | Literal['all', 'no_enums'] | None*)
- **_cli_shortcuts** (*Mapping[str, str | list[str]] | None*)
- **_secrets_dir** (*PathType | None*)
- **_build_sources** (*tuple[tuple[PydanticBaseSettingsSource, ...], dict[str, Any]] | None*)
- **app_name** (*str*)
- **version** (*str*)
- **environment** (*str*)
- **debug** (*bool*)
- **log_level** (*str*)
- **source** (*str*)
- **topic** (*str*)
- **quality** (*int*)
- **frame_width** (*int*)
- **frame_height** (*int*)
- **framerate** (*int*)
- **hfov** (*float*)
- **vfov** (*float*)
- **fx** (*float*)
- **fy** (*float*)
- **cx** (*float*)
- **cy** (*float*)
- **calibration_width** (*int*)
- **calibration_height** (*int*)
- **relative_to** (*str*)

SpiriCamera.main.**setup_logging**(*level='INFO'*)

Configure loguru logging.

Parameters

level

[str] Logging level string (e.g. DEBUG, INFO).

Parameters

level (*str*)

Return type

None

SpiriCamera.main.**get_settings**()

Load settings from environment.

Returns

Settings

Configured settings instance.

Return type

Settings

SpiriCamera.main.**main**()

Main entry point.

Return type

None

- *SpiriCamera.camera* — capture lifecycle, encoding, and publication
- *SpiriCamera.exif* — frame metadata carried inside the encoded frame
- *SpiriCamera.overlay* — SVG HUD overlays baked into frames before encoding
- *SpiriCamera.projection* — where a point in the world lands in a camera's picture
- *SpiriCamera.sources* — source handler registry and resolution
- *SpiriCamera.webrtc* — WebRTC plumbing shared by every signaling path
- *SpiriCamera.whep* — WHEP (WebRTC-HTTP Egress Protocol) exposure for a single camera
- *SpiriCamera.ui* — NiceGUI debug UI
- *SpiriCamera.web* — web UI entry point (optional SpiriConfig plugin)
- *SpiriCamera.cli* — command-line interface
- *SpiriCamera.main* — main entry point

CHAPTER

SEVEN

REVISION HISTORY

This table is generated from `revisions.yaml` at the root of this repository. Do not edit it here – add an entry there and it appears below, on the cover page, and in the PDF footer.

Revision	Date	Summary	Author
0.1 (draft)	2026-01-01	Initial draft.	Alex Davies

7.1 Issuing a revision

The document's revision is separate from the software's version: revise the document when what it says changes.

1. Add an entry to the bottom of `revisions.yaml`, without `draft: true`.
2. Merge it. The next software release (`vX.Y.Z` tag) attaches the PDF of this revision to its release. That release asset is the controlled copy: link people to it rather than to a PDF built from a branch, which can change under them.

PYTHON MODULE INDEX

S

SpiriCamera.camera, 58
SpiriCamera.cli, 113
SpiriCamera.exif, 72
SpiriCamera.main, 115
SpiriCamera.overlay, 75
SpiriCamera.projection, 88
SpiriCamera.sources, 91
SpiriCamera.ui, 109
SpiriCamera.web, 113
SpiriCamera.webrtc, 106
SpiriCamera.whep, 108

INDEX

A

add_metrics_widget() (*in module SpiriCamera.ui*), 109
add_tiger_widget() (*in module SpiriCamera.ui*), 110
aliases (*SpiriCamera.overlay.WorldPlacement property*), 81
anchor (*SpiriCamera.overlay.HudWidget attribute*), 78
anchor (*SpiriCamera.overlay.WorldPlacement attribute*), 81
anchor_position() (*in module SpiriCamera.overlay*), 82
ANCHORS (*in module SpiriCamera.overlay*), 76
app_name (*SpiriCamera.main.Settings attribute*), 117

B

build_page() (*in module SpiriCamera.ui*), 110

C

calibration_height (*SpiriCamera.main.Settings attribute*), 118
calibration_width (*SpiriCamera.main.Settings attribute*), 118
Camera (*class in SpiriCamera.camera*), 64
camera_metrics_widget() (*in module SpiriCamera.overlay*), 82
CameraBase (*class in SpiriCamera.camera*), 59
CameraError, 59
CameraNotStartedError, 59
capture() (*in module SpiriCamera.cli*), 114
capture_params() (*SpiriCamera.sources.OpenCVSource method*), 94
capture_settings() (*SpiriCamera.camera.Camera method*), 68
capture_target() (*SpiriCamera.sources.OpenCVSource method*), 94
CaptureSettings (*class in SpiriCamera.sources*), 91
clear_intrinsics() (*SpiriCamera.camera.Camera method*), 68
close() (*SpiriCamera.camera.Camera method*), 66
close() (*SpiriCamera.sources.OpenCVSource method*), 95
close() (*SpiriCamera.sources.SourceBase method*), 98
close_peer_sync() (*in module SpiriCamera.webrtc*), 106
close_session_async() (*in module SpiriCamera.webrtc*), 107
close_session_sync() (*in module SpiriCamera.webrtc*), 107
composite() (*in module SpiriCamera.overlay*), 83
create_overlay_widget() (*in module SpiriCamera.ui*), 110
create_whep_app() (*in module SpiriCamera.whep*), 108
custom_anchor_x (*SpiriCamera.overlay.HudWidget attribute*), 78
custom_anchor_y (*SpiriCamera.overlay.HudWidget attribute*), 78
cx (*SpiriCamera.main.Settings attribute*), 118
cx (*SpiriCamera.sources.Intrinsics attribute*), 92
cy (*SpiriCamera.main.Settings attribute*), 118
cy (*SpiriCamera.sources.Intrinsics attribute*), 92

D

DATETIME_TAG (*in module SpiriCamera.exif*), 72
debug (*SpiriCamera.main.Settings attribute*), 117
declared_objects() (*in module SpiriCamera.overlay*), 84
DEFAULT_FONT (*in module SpiriCamera.overlay*), 76
DEFAULT_FONT_PATH (*in module SpiriCamera.overlay*), 76

describe() (*SpiriCamera.sources.SourceBase method*), 97
describe_capabilities() (*SpiriCamera.camera.Camera method*), 68
describe_source() (*in module SpiriCamera.sources*), 104
device_metadata() (*SpiriCamera.sources.OpenCVSource method*), 94
discover_cameras() (*in module SpiriCamera.ui*), 111
discover_widgets() (*in module SpiriCamera.ui*), 111

E

each (*SpiriCamera.overlay.WorldPlacement attribute*), 81
edge (*SpiriCamera.projection.View attribute*), 89
embed() (*in module SpiriCamera.exif*), 73
ENCODINGS (*in module SpiriCamera.camera*), 58
environment (*SpiriCamera.main.Settings attribute*), 117
error (*SpiriCamera.sources.SourceInfo attribute*), 102
exif_clear_tags() (*SpiriCamera.camera.Camera method*), 70
exif_enabled (*SpiriCamera.camera.CameraBase attribute*), 63
EXIF_NAME (*in module SpiriCamera.overlay*), 77
exif_set_tags() (*SpiriCamera.camera.Camera method*), 70
exif_tag_providers() (*SpiriCamera.camera.Camera method*), 70
exif_tags (*SpiriCamera.camera.Camera property*), 69
exif_tags_for_frame() (*SpiriCamera.camera.Camera method*), 71
exif_timestamp (*SpiriCamera.camera.Camera property*), 69
exif_update() (*SpiriCamera.camera.Camera method*), 71
ExifError, 73
extract() (*in module SpiriCamera.exif*), 73

F

frame (*SpiriCamera.sources.SourceCapabilities attribute*), 101
frame_height (*SpiriCamera.main.Settings attribute*), 117
frame_intrinsics() (*SpiriCamera.sources.SourceBase method*), 99
FRAME_NAME (*in module SpiriCamera.overlay*), 77
FRAME_ROUTE (*in module SpiriCamera.ui*), 109
frame_tags() (*SpiriCamera.sources.SourceBase method*), 99
frame_width (*SpiriCamera.main.Settings attribute*), 117
framerate (*SpiriCamera.main.Settings attribute*), 117
FrameUnavailableError, 59
from_fov() (*SpiriCamera.sources.Intrinsics class method*), 92
from_url() (*SpiriCamera.sources.SourceBase class method*), 97
fx (*SpiriCamera.main.Settings attribute*), 117
fx (*SpiriCamera.sources.Intrinsics attribute*), 92
fy (*SpiriCamera.main.Settings attribute*), 118
fy (*SpiriCamera.sources.Intrinsics attribute*), 92

G

get_camera() (*in module SpiriCamera.ui*), 111
get_loop() (*in module SpiriCamera.webrtc*), 107
get_settings() (*in module SpiriCamera.main*), 120

H

handler (*SpiriCamera.camera.Camera property*), 66
handler (*SpiriCamera.sources.SourceInfo attribute*), 102
handles() (*SpiriCamera.sources.SourceBase class method*), 96
height (*SpiriCamera.sources.Intrinsics attribute*), 92
hfov (*SpiriCamera.main.Settings attribute*), 117
hfov (*SpiriCamera.sources.Intrinsics property*), 93
HudWidget (*class in SpiriCamera.overlay*), 77

I

image (*SpiriCamera.camera.CameraBase attribute*), 63
image_size() (*in module SpiriCamera.exif*), 74
Intrinsics (*class in SpiriCamera.sources*), 91
intrinsics (*SpiriCamera.camera.CameraBase attribute*), 62
intrinsics (*SpiriCamera.sources.SourceCapabilities attribute*), 101

intrinsics() (*SpiriCamera.main.Settings method*), 118
INTRINSICS_ORIGINS (*in module SpiriCamera.camera*), 58
is_jpeg() (*in module SpiriCamera.exif*), 74
is_open (*SpiriCamera.sources.OpenCVSource property*), 95
is_open (*SpiriCamera.sources.SourceBase property*), 98
ITEM_NAME (*in module SpiriCamera.overlay*), 77

K

known_schemes() (*in module SpiriCamera.sources*), 105

L

log_level (*SpiriCamera.main.Settings attribute*), 117

M

main() (*in module SpiriCamera.main*), 120
main_callback() (*in module SpiriCamera.cli*), 113
max_framerate (*SpiriCamera.camera.CameraBase attribute*), 63
max_framerate (*SpiriCamera.sources.CaptureSettings attribute*), 91
max_height (*SpiriCamera.camera.CameraBase attribute*), 63
max_height (*SpiriCamera.sources.CaptureSettings attribute*), 91
max_supported_framerate (*SpiriCamera.camera.CameraBase attribute*), 62
max_supported_framerate (*SpiriCamera.sources.SourceCapabilities attribute*), 101
max_supported_height (*SpiriCamera.camera.CameraBase attribute*), 62
max_supported_height (*SpiriCamera.sources.SourceCapabilities attribute*), 101
max_supported_width (*SpiriCamera.camera.CameraBase attribute*), 62
max_supported_width (*SpiriCamera.sources.SourceCapabilities attribute*), 101
max_width (*SpiriCamera.camera.CameraBase attribute*), 63
max_width (*SpiriCamera.sources.CaptureSettings attribute*), 91
mimetype (*SpiriCamera.camera.CameraBase attribute*), 63
model (*SpiriCamera.camera.CameraBase attribute*), 62
model (*SpiriCamera.sources.SourceCapabilities attribute*), 101
model_config (*SpiriCamera.main.Settings attribute*), 116
module
    SpiriCamera.camera, 58
    SpiriCamera.cli, 113
    SpiriCamera.exif, 72
    SpiriCamera.main, 115
    SpiriCamera.overlay, 75
    SpiriCamera.projection, 88
    SpiriCamera.sources, 91
    SpiriCamera.ui, 109
    SpiriCamera.web, 113
    SpiriCamera.webrtc, 106
    SpiriCamera.whep, 108

N

NEAR (*in module SpiriCamera.projection*), 88
negotiate_async() (*in module SpiriCamera.webrtc*), 107
negotiate_sync() (*in module SpiriCamera.webrtc*), 107

O

OBJECTS_NAME (*in module SpiriCamera.overlay*), 77
open() (*SpiriCamera.sources.OpenCVSource method*), 95
open() (*SpiriCamera.sources.SourceBase method*), 98
OpenCVSource (*class in SpiriCamera.sources*), 94
origin (*SpiriCamera.sources.Intrinsics attribute*), 92
overlay_client_render (*SpiriCamera.overlay.OverlayMixin attribute*), 80
overlay_widgets (*SpiriCamera.overlay.OverlayMixin attribute*), 79
OverlayError, 79
OverlayMixin (*class in SpiriCamera.overlay*), 79

P

parse() (*SpiriCamera.sources.SourceURL class method*), 103

pixel() (*in module SpiriCamera.projection*), 90
project (*SpiriCamera.overlay.WorldPlacement attribute*), 81
project() (*in module SpiriCamera.projection*), 90
pull_sync() (*in module SpiriCamera.webrtc*), 108

Q

quality (*SpiriCamera.camera.CameraBase attribute*), 63
quality (*SpiriCamera.main.Settings attribute*), 117

R

r (*SpiriCamera.projection.View attribute*), 89
range (*SpiriCamera.projection.View attribute*), 89
rasterize() (*in module SpiriCamera.overlay*), 84
raw (*SpiriCamera.sources.SourceURL attribute*), 103
read() (*SpiriCamera.camera.Camera method*), 69
read() (*SpiriCamera.sources.OpenCVSource method*), 95
read() (*SpiriCamera.sources.SourceBase method*), 98
read_frame() (*SpiriCamera.camera.Camera method*), 68
received_framerate (*SpiriCamera.camera.CameraBase attribute*), 63
received_height (*SpiriCamera.camera.CameraBase attribute*), 62
received_ratio (*SpiriCamera.camera.CameraBase attribute*), 62
received_width (*SpiriCamera.camera.CameraBase attribute*), 62
registered_sources() (*in module SpiriCamera.sources*), 105
relative_to (*SpiriCamera.main.Settings attribute*), 118
remove_overlay_widget() (*in module SpiriCamera.ui*), 112
render_overlays_for_client() (*SpiriCamera.overlay.OverlayMixin method*), 80
render_svg() (*in module SpiriCamera.overlay*), 85
resolve_objects() (*in module SpiriCamera.overlay*), 86
resolve_source() (*in module SpiriCamera.sources*), 105
run() (*in module SpiriCamera.cli*), 114
running (*SpiriCamera.camera.CameraBase attribute*), 63

S

scaled() (*SpiriCamera.sources.Intrinsics method*), 93
scheme (*SpiriCamera.sources.SourceBase property*), 97
scheme (*SpiriCamera.sources.SourceInfo attribute*), 102
scheme (*SpiriCamera.sources.SourceURL attribute*), 103
schemes (*SpiriCamera.sources.SourceBase attribute*), 96
SDP_MEDIA_TYPE (*in module SpiriCamera.whep*), 108
serial_number (*SpiriCamera.camera.CameraBase attribute*), 62
serial_number (*SpiriCamera.sources.SourceCapabilities attribute*), 101
serve_frame() (*in module SpiriCamera.ui*), 112
set_authoritive() (*in module SpiriCamera.ui*), 112
set_intrinsics() (*SpiriCamera.camera.Camera method*), 67
set_mirror() (*in module SpiriCamera.ui*), 113
Settings (*class in SpiriCamera.main*), 115
setup_logging() (*in module SpiriCamera.main*), 119
source (*SpiriCamera.camera.CameraBase attribute*), 62
source (*SpiriCamera.main.Settings attribute*), 117
source_str (*SpiriCamera.camera.CameraBase attribute*), 62
SourceBase (*class in SpiriCamera.sources*), 96
SourceCapabilities (*class in SpiriCamera.sources*), 100
SourceError, 101
SourceInfo (*class in SpiriCamera.sources*), 101
sources() (*in module SpiriCamera.cli*), 114
SourceURL (*class in SpiriCamera.sources*), 103
SpiriCamera.camera
    module, 58
SpiriCamera.cli
    module, 113
SpiriCamera.exif
    module, 72
SpiriCamera.main
    module, 115

SpiriCamera.overlay
module, 75
SpiriCamera.projection
module, 88
SpiriCamera.sources
module, 91
SpiriCamera.ui
module, 109
SpiriCamera.web
module, 113
SpiriCamera.webrtc
module, 106
SpiriCamera.whep
module, 108
STANDARD_TAGS (*in module SpiriCamera.exif*), 72
start() (*SpiriCamera.camera.Camera method*), 66
status (*SpiriCamera.camera.CameraBase attribute*), 63
STATUS_RUNNING (*in module SpiriCamera.camera*), 58
STATUS_STOPPED (*in module SpiriCamera.camera*), 58
STATUS_WAITING (*in module SpiriCamera.camera*), 58
stop() (*SpiriCamera.camera.Camera method*), 66
svg_template (*SpiriCamera.overlay.HudWidget attribute*), 78
synq_skip_sync (*SpiriCamera.camera.CameraBase attribute*), 62

T

TAGS_CHANNEL_LABEL (*in module SpiriCamera.webrtc*), 106
target (*SpiriCamera.sources.SourceBase property*), 97
target (*SpiriCamera.sources.SourceInfo attribute*), 102
target (*SpiriCamera.sources.SourceURL attribute*), 103
tiger_widget() (*in module SpiriCamera.overlay*), 87
TIMESTAMP_TAG (*in module SpiriCamera.exif*), 72
to_camera() (*in module SpiriCamera.projection*), 90
topic (*SpiriCamera.main.Settings attribute*), 117
topic_for_source() (*in module SpiriCamera.camera*), 59

U

UnknownSourceError, 104
unresolved() (*SpiriCamera.sources.SourceInfo class method*), 102
update() (*SpiriCamera.sources.SourceInfo method*), 102
url (*SpiriCamera.sources.SourceInfo attribute*), 102

V

validate() (*in module SpiriCamera.cli*), 114
vendor (*SpiriCamera.camera.CameraBase attribute*), 62
vendor (*SpiriCamera.sources.SourceCapabilities attribute*), 101
version (*SpiriCamera.main.Settings attribute*), 117
version() (*in module SpiriCamera.cli*), 113
vfov (*SpiriCamera.main.Settings attribute*), 117
vfov (*SpiriCamera.sources.Intrinsics property*), 93
View (*class in SpiriCamera.projection*), 89
VIEW_NAME (*in module SpiriCamera.overlay*), 77
visible() (*SpiriCamera.projection.View method*), 89

W

webrtc_offer() (*SpiriCamera.camera.Camera method*), 66
whep_serve() (*in module SpiriCamera.cli*), 114
width (*SpiriCamera.sources.Intrinsics attribute*), 92
world_placement() (*in module SpiriCamera.overlay*), 88
WorldPlacement (*class in SpiriCamera.overlay*), 81

X

x (*SpiriCamera.projection.View attribute*), 89

Y

y (*SpiriCamera.projection.View attribute*), 89

Z

z_index (*SpiriCamera.overlay.HudWidget attribute*), 78